UNIVERSITÀ
DI TRENTO

Department of Information Engineering and Computer Science

Bachelor's Degree in
Computer, Communication and Electronic Engineering

Final Dissertation

TinyML-Based Voice Recognition System on Syntiant NDP101

From Keyword Spotting to Speaker Verification

Supervisor
Prof. Kasim Sinan Yildirim

Student
Matteo Gottardelli

Academic year 2024/2025

Contents

Abstract

This thesis focuses on adapting a TinyML device-based system to perform Speaker Verification, which involves recognizing a user's identity by comparing reference samples with an input audio stream, using a neural network trained on a personal created dataset. The main objectives were to create a Keyword Spotting model and a Speaker Verification text-dependent one, adapting their dimension to fit inside a TinyML device, specifically the Syntiant NDP101. The methods used in the study included neural network training, using Edge Impulse framework for model compression, validation, and deployment for Keyword Spotting, and a d-vector extractor technique to develop a Speaker Verification one. The key findings include the development of a system which in theory can be deployed on a TinyML device; however because of an NDA on Syntiant NDP101 it could not be verified on hardware, but only on software. It was underlying during the thesis the importance of output size in achieving better performance, with increasing representational capacity, leading in proposing d-vector model's versions. It was created a software C logic to emulate audio MFE block processing, models behavior and a proposed distillation knowledge algorithm for Syntiant NDP101 that does not support Convolutional Neural Networks, but only Dense ones. The results of the study showed which of the models developed and distillated are deployable on the MCU, in particular a Keyword Spotting Model, 2 d-vector extractors and 5 distilled models. From this thesis can be acquired the processed developed to create and deploy a model on a TinyML device and an in-depth on their performances in terms of accuracy, precision, recall, EER, and AUC from a general purpose perspective, choosing a threshold that minimizes EER value, and a security one, which has a precision equal to 1. The limitations include the limited complexity of the neural network deployable on Syntiant NDP101 and some careful considerations of the computation velocity, memory usage, precision, recall, true positive rate and false positive rate. The future work includes exploring the use of other neural networks or techniques to optimize the generated models to achieve better results, especially in distilled versions and an actual deployment on Syntiant (if the access to SDK is granted); in the other case, it could be a possibility to switch to another platform with similar properties. The usage of such system may be for general purpose application, like voice assistants or smart home devices, or security ones, allowing less energy consumption even though it is an always-on chip.

1 Introduction

Microcontrollers (MCUs) are computing systems that are integrated into a larger ecosystem; in this case, they acquire the adjective Embedded. This means that they are designed to perform a specific function that requires a software implementation (programmed in C or assembly) and a hardware implementation (interconnectivity wires and sensor handle). They operate in a closed environment and elaborate the physical inputs, which may be visual, acoustic, and with more recent technology even tactile or movement-based. These are elaborated to generate an output, which may be feedback to a larger system, an audio response, or a trigger depending on the microcontroller specifics. The programmer can implement a desired application for real-time use, allowing personalized functionalities and optimizing possibilities. In application development, the objective is to achieve cost reduction in power and energy terms and the opportunity to build a desired application, adding more than one feature at the same time, and connecting various sensors thanks to their peripherals.

1.1 TinyML Concept and Limits

MCU's technology made steps ahead in optimizing the computation velocity, meanwhile minimizing power consumption, and a result is TinyML (Tiny Machine Learning). These devices enable machine and deep learning models to operate on an MCU, allowing performing actions like Keyword Spotting, recognizing a specific word in an audio stream, or identifying objects in an image. These functionalities can be implemented thanks to a Neural Network, which is typically trained on cloud resources in the Python programming language, and this leads to performing only the inference phase on these tiny devices. This approach does not allow data exploitation directly, limiting incremental training or adapting algorithms through the device's life. This is a limit on the Machine Learning side, but on the tiny one, there are some trade-offs. A direct consequence of being too small devices is having limited memory to reduce power consumption, so sometimes adapting a neural network, which typically may occupy much memory, is not easy and requires precision reduction.

1.2 Goals - TinySV

This thesis studies how to adapt a TinyML device's system based on an application that performs Speaker Verification, whose task consists of recognizing the identity of a user with reference samples and comparing them with a processed input audio stream. The objective originally was creating a Keyword Spotting model (KWS) and a Speaker Verification (SV) one, trying to adapt that algorithm on two Syntiant TinyML NDP101 devices, but because of an NDA, the access to documentation was inaccessible. The KWS development was possible thanks to Edge Impulse[13], but the SV approach[22] using dvectors is a technique that is not supported by Edge Impulse and due to the inability to access a model compression tool. To preserve the initial idea, the model was tested as it would be on a Syntiant TinyML NDP101, to show the validation of the technique. The objective of this thesis is to show the feasibility of this idea from a software perspective and to demonstrate if deployability on the model is possible. All codes used in this thesis are provided on a GitHub repository[1][9].

1.3 Brief Summary

The thesis is divided into chapters. After this introduction, Chapter 2 aims to present theoretical concepts that will be used in this thesis, like Audio Processing, Deep Learning concepts, KWS, and SV. Chapter 3 explains the workflow of the model's training techniques adopted. Chapter 4 introduces the general methodology of the final objective and explains how it works. Chapter 5 draws up the results obtained from computer testing code, which emulates Syntiant NDP101 behavior. Chapter 6 contains the thesis conclusion and future possible work.

[1]Thesis GitHub Repository - https://github.com/Gotta003/Syntiant-NDP101-Speaker-Verification-Thesis

2 Relevant Theoretical Notions

2.1 NDP101 Architecture Overview

The NDP101 is a microcontroller unit (MCU) developed by Syntiant[24], a company that specializes in the development of edge AI devices. "NDP" stands for neural decision processor, an architecture used for deep learning algorithms applied to audio processing applications, such as keyword speech interface, sensor recognition, and speaker identification. The device is composed of two components:
1. TinyBoard: Contains the CPU that handles the peripherals and contains the hardware part.
2. Syntiant NDP101 Core: In this part, all the audio processing happens, and it is where the neural network is stored. It is relevant that the DNN (Dense Neural Network)[2] architecture, which will be stored in the device, is fixed. 256KB are dedicated with int32 bias length and int4 weights[34], which can be at most 589.000 total parameters in that memory location. The Neural Network for this device to be fast enough in computation avoids the use of CNN (Convolutional Neural Network)[20], and it can only support four Fully Connected Layers, three intermediate with 256 neurons, and one for output with at most 64 output classes, using classification. To perform internal software computation, there is an internal SRAM inside the chip, which is a Cortex-M0 112KB size. This piece of memory contains the binary user's code along with global and local variables[6][24][25].

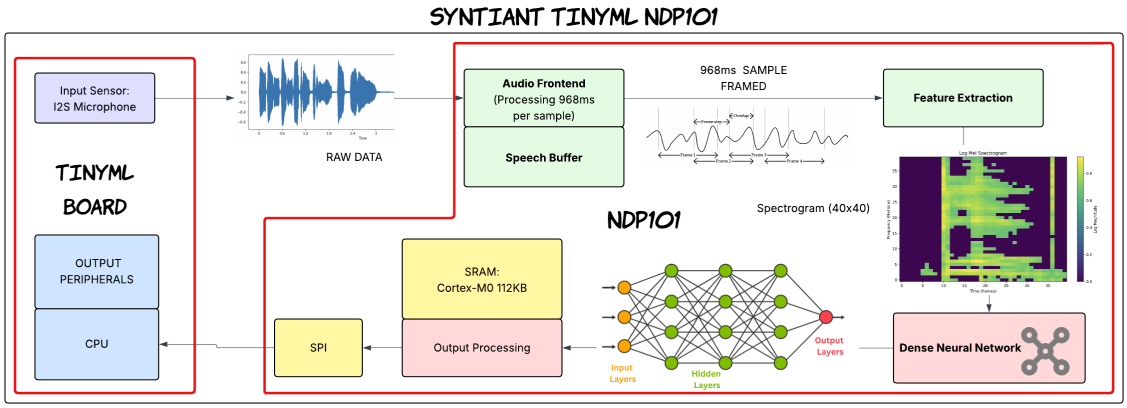


Figure 2.1: Syntiant NDP101 High-Level Workflow

A typical device behavior follows the workflow in **Figure 2.1**. It involves the use of an already integrated I2S microphone as an input sensor. If the MCU is powered and it is activated, it will be an always on data capture, performing a polling behavior. The input is processed by the Audio Front End, which will handle samples of 968ms, while the system feeds a 96KB speech buffer[30]. These samples are given as input to the feature extraction module, which will act as MFE Block Processing[12], using a log-mel spectrogram. This spectrogram will always have 1600 features, which corresponds to a 40x40 spectrogram image. Then, this generated image is processed by the Dense Neural Network. Although classification is the default behavior, developers can customize how outputs are handled using a custom Arduino IDE code[5].

2.1.1 Device Peripherals

In addition to the microphone input, the board includes:
• 9 pins: 4 for the power supply and 5 for the GPIO (general purpose I/O)
• One Serial Flash Memory (SRAM)
• One micro-SD card slot used for memory extension
The last component is used for big data storage and the IMU functionality, which is supported by the

device. It is estimated that with a 32GB card, it will save more than 3 days of uncompressed audio data, using a frequency of 16kHz, and with IMU more than 300 days of 6-axis sensor data using a frequency of 100Hz[3]. In **Figure 2.2**, the peripheral connections on Syntiant NDP101 are shown:

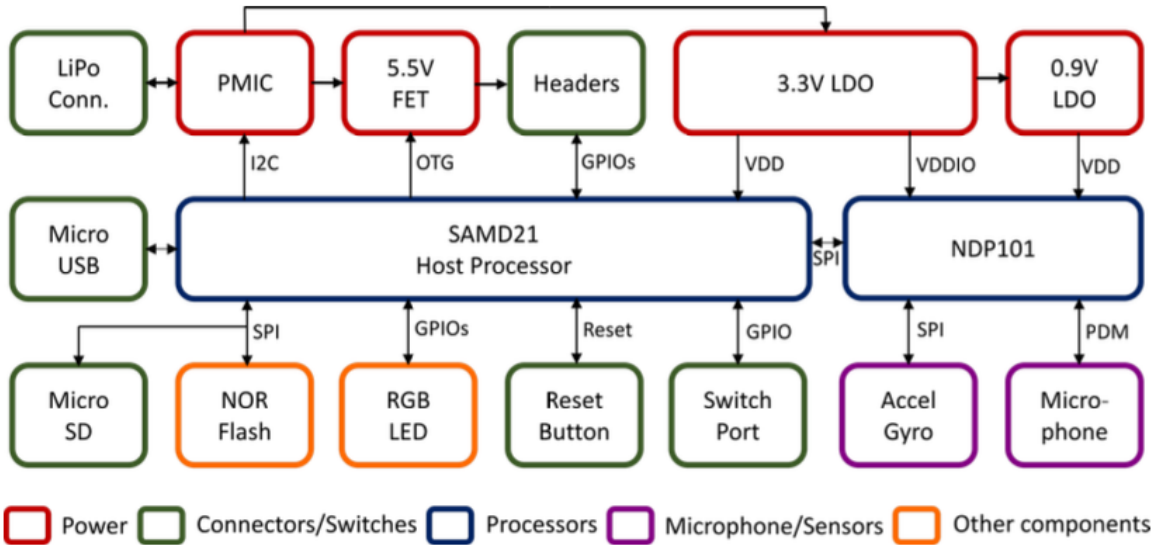


Figure 2.2: Design NDP101 with peripherals [3]

Talking about power metrics, NDP101, because it is an always-on power consumption device, has the capacity to use audio/voice recognition applications at 140μW. These results compared to other MCUs will produce 20 times more throughput and 200 times less energy per inference[3]. The connection with another MCU is possible with SPI communication; however, without the SDK provided by Syntiant, it is challenging to implement that.

2.2 Syntiant Audio Block Processing

Edge Impulse reports that the Syntiant Audio Block Processing is similar to that of the MFE one[12]. Its goal is to extract time and frequency features from a raw audio input. However, the block processing of Syntiant defers a little, because of a noise floor at the end of the computation. The block, which corresponds to the feature extractor, can be viewed in **Figure 2.3**[26][16]:

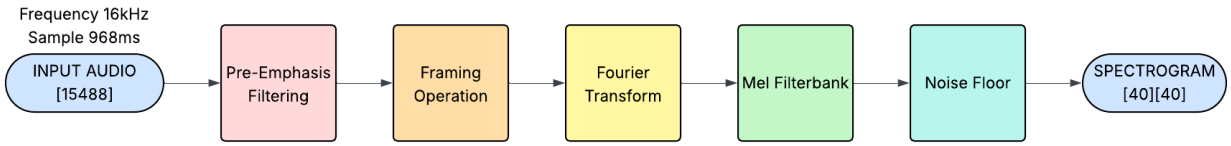


Figure 2.3: Syntiant Audio Block Processing

1. Input - The frequency input of raw data is at 16kHz and sampling at 968ms. The number of raw data=$frequency \cdot sample\,length$ generates 15488 raw data, with a short value size because the audio sound can go from -2^{15} to $2^{15}-1$. These values come from an implicit ADC (Analog Digital Conversion), using a dual PDM microphone input, which interfaces with an I2S interface multiplexed with PDM[29]. It stands for pulse density modulation and reduces the system into a single-bit digital one. This allows signal processing operations to be easily performed on the audio stream, and then the PDM can be manipulated by the system.
2. Pre-emphasis filter - This is a high-pass filter that enhances high-frequency components, so the microphone will capture more low-frequency noise and increase high frequencies to make the speech clearer. Before this, an audio normalization [-1, 1] is required to generate float values and then to

apply this high-filter:

$$y[n] = x[n] - \alpha \cdot x[n-1] \tag{2.1}$$

x is the input signal, y the output signal and α is a filter grade coefficient. Syntiant block set at 0.96875 by default.

3. Framing - This audio is split and segmented into small overlapping windows called frames. Each one has an overlap time with each other and on the Syntiant device corresponds to 128 floats. It is derived subtracting frame's size of 512 floats and stride's one of 384 corresponding to how much the start position will move. In the last frame, a part will overflow the initial buffer, and in that case, the void values are flattened to 0. Considering the input of 15488 samples in a 16kHz frequency:

$$number\,of\,frames = \frac{input\,size - frame\,size}{frame\,stride} + 1 = \frac{15488 - 512}{384} + 1 = 40 \tag{2.2}$$

For each frame of the forty computed, the following operations are performed:

3.1 Windowing - Before performing the Fourier transform, it has to be applied a windowing to reduce spectral leakage in integration, so the following sinusoidal function is used:

$$0.54 - 0.46 \cdot (\frac{2\pi k}{size - 1}) \tag{2.3}$$

The size is 512, k is an incremental value that goes from 0 up to 511 and k's window is multiplied with the k's position of the array.

3.2 Fast Fourier Transformation (FTT) - This function computes the complex frequency spectrum of a real-valued signal captured in the time domain. It is not required to compute all the DFT domain because, dealing with real values using Hermitian symmetry property with real values, it requires to compute only $\frac{N}{2} + 1$ unique complex outputs.

$$\begin{cases} X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{\frac{-2\pi i k n}{N}} & k = 0, 1, ..., \frac{N}{2} \\ X[l] = \overline{X[k]} & l = N - k \end{cases} \tag{2.4}$$

x[n] is the input signal with n=0,...,N-1, x[n]$\in \mathbb{R}$, i is the imaginary unit, k is the incremental value, and N is the length of the signal (512 values).

3.3 Spectrogram Population - Corresponds to a magnitude computation of the FFT output, obtaining the amplitude spectrum from the complex frequency-domain data. The operation computes the norm of the first half of the Fourier transformation output and saves it in the corresponding size [40x256], the magnitude is computed as follows:

$$|X[k]| = \sqrt{(Re(X[k]))^2 + (Im(X[k]))^2},\ k = 0, 1, ..., \frac{N}{2} - 1 \tag{2.5}$$

4. Mel filterbank - After obtaining this matrix, the algorithm applies a mel filterbank, which is a set of data based on the human perception system via a triangular bandpass filter, making the system more sensitive to low frequencies. It is used mel scale to obtain this phenomenon because using a logarithmic approach allows better sound recognition.

This implements a K+2 length filter called m, composed of linearly spaced elements, with K being the number of filters.

$$m[i] = M_{min} + i \cdot \frac{M_{max} - M_{min}}{K + 1},\ i = 0, 1, ..., K - 1 \tag{2.6}$$

M_{min} and M_{max} correspond to the conversion in the mel scale of the minimum and the maximum frequency. Syntiant as default has set the minimum at 0 and the maximum at $\frac{f_s}{2}$. The information is then reconverted back to the frequency scale. The formulas for Syntiant are as follows:

$$m = 1127 \cdot log_{10}(1 + \frac{h}{700}) \quad h = 700 \cdot 10^{\frac{m}{1127}} - 1 \tag{2.7}$$

To this scale, a triangular filter for each filter between 1 and K is applied, using bins that cover frequencies. The computation of the bins and the triangular function is the following:

$$b_i = \lfloor \frac{2 \cdot f_i}{f_s} \cdot (N-1) \rfloor \quad H_k[b] = \begin{cases} 0 & b < b_k - 1 or b > b_k + 1 \\ \frac{b - b_{k-1}}{b_k - b_{k-1}} & b_{k-1} \leq b \leq b_k \\ \frac{b_{k+1} - b}{b_{k+1} - b_k} & b_k \leq b \leq b_{k+1} \end{cases} \tag{2.8}$$

This computation creates a matrix 40x40, which corresponds to the filterbank filter that will be applied to the magnitudes matrix to obtain a log-mel spectrogram:

$$L(i,j) = 10 \cdot log_{10}(\sum_{k=0}^{NUM\_BINS-1} S(i,k) \cdot M(j,k) + \epsilon) \;\; i = 0, ..., N-1 \;\; j = 0, ..., F-1 \;\; \epsilon = 10^{-20} \tag{2.9}$$

The computation will generate a 40x40 matrix that will be the form of the image to be fed into the neural network. To the computation a small constant error value is added to avoid the log(0) result, L is the log-mel spectrogram, M is the fixed filterbank and S is the spectrogram of the magnitudes
5. Noise Floor - To only maintain audible sound is applied a threshold noise floor flatting all normalized values below 0.65, which should be resized according to the noise floor in decibels of the system, which for Syntiant is set to -40dB. The formula for doing this is the following:

$$final(i,j) = \frac{L(i,j) - NOISE\,FLOOR}{-NOISE\,FLOOR + 12} \;\; i,j = 0, ..., FILTERS \tag{2.10}$$

To be accepted, the final(i,j) should be ≥ 0.65 and the overall final matrix is the input given to the neural network.

2.3 Neural Network Concept

2.3.1 Dense Neural Network (DNN)

Neural networks consist of interconnected layers organized in a logical architecture, having inside each layer a predecided number of neurons[2]. These are like nodes that connect two nearby layers. Each receives an input signal and applies to it an activation function, generating an intermediate output that will be passed to the subsequent series of neurons to the other layer. The connections between neurons are weighted, so there are some values associated with the layer that represent the influence of these connections that will modify the input to generate an output, as in **Figure 2.4**. The weights can be adjusted only during the training phase, and they characterize the neural network behavior, picking a layer as reference that patterns the connection of each input with each output, generating an allocation in memory equal to "input size· output size". Typically, a bias array is associated with each layer, always modifiable only during training, which performs an adjustment to the output of the neuron. It is like a simple addition, so for each, the array is as big as the output size to have an adjustment value for each output.

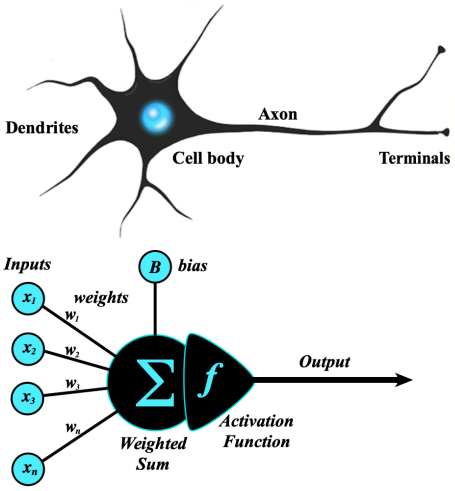


Figure 2.4: Neuron structure [31]

The neuron can be defined as a formula consisting of an input matrix x[N]=$[x_1, ..., x_N]^T$ as Nx1, the weights that involve only that output neuron, because it is a large array with size "in_dimension·out_dimension" and we can express it as matrix w[M]=$[w_1, ..., w_M]^T$ Mx1, b the bias associated, an adjustment value per neuron, $\phi(\cdot)$ the activation function and y as the output of the

neuron.

$$y = \phi(w^T x + b) = \phi(\sum_{i=1}^{n} w_i x_i + b) \tag{2.11}$$

More generally, the formula that will represent the output array of a layer is computed considering W$\in \mathbb{R}^{O \times N}$ as the OxN matrix, the biases b=$[b_1, ..., b_O]^T$ as Ox1 matrix and the output as y=$[y_1, ..., y_O]^T$ as Ox1 matrix, resulting in:

$$y_i = \phi(Wx + b) = \phi(\sum_{i=1}^{n} w_i j x_i + b_i) \; i = 1, ..., O \rightarrow \begin{cases} y_1 = \phi(w_{11}x_1 + w_{12}x_2 + ... + w_1 N x_n + b_1) \\ y_2 = \phi(w_{21}x_1 + w_{22}x_2 + ... + w_2 N x_n + b_2) \\ \dots \\ y_O = \phi(w_{O1}x_1 + w_{O2}x_2 + ... + w_O N x_n + b_O) \end{cases} \tag{2.12}$$

The architecture of the neural network is made up of different layers according to their functionality principles, like in **Figure 2.5**. The most notable ones are:

• Input layer: Consists in the input of the system in the case of Syntiant neural network a spectrogram 40x40 flatted into a 1600 feature array. This layer is only nominal, and it does not perform any computations.

• Intermediate layer: A neural network can have one or more of these that are between the input and the output layers. The decision of the number of these depends on the memory space and application specifics, but typically more layers equal to a more complex network structure. It is characterized by an activation function typically a ReLU. On the Syntiant NDP101 device, there are 3 intermediate layers, each with 256 neurons.

• Output layer: Consists in the final output of the network and the output depends on the behavior of the network; if it is a regression-like model it will output inside a neuron a value using an activation ReLU, but if it performs a classification, it means it is a multiclass model and in that case the softmax activation to identify the probability of belonging in each class. Syntiant devices can have at most 64 neurons as output, with the above neuron configuration.

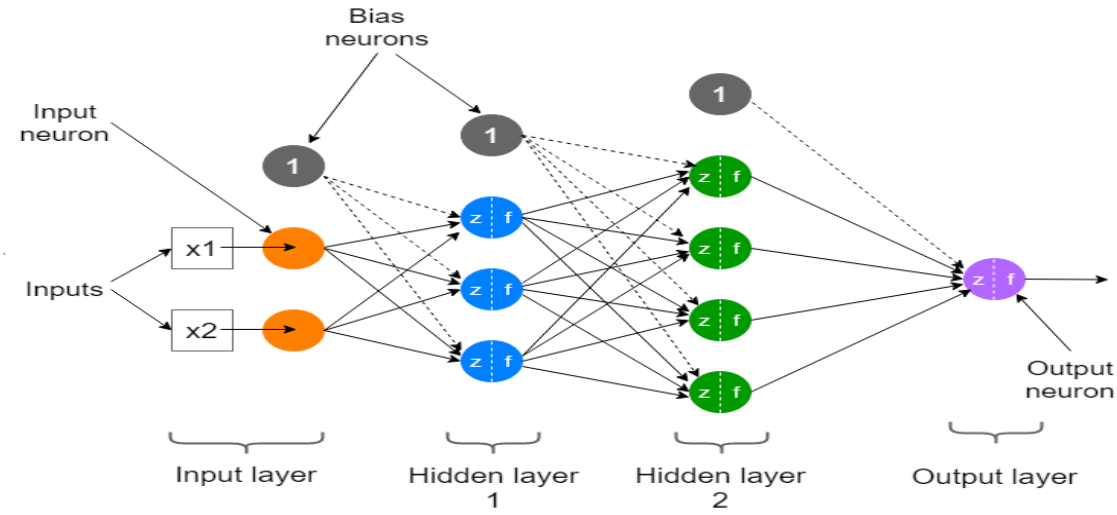


Figure 2.5: Neural Network Structure [23]

For this thesis, we are interested in only two activation functions, already nominated:

• ReLU (Rectified Linear Unit): Performs a threshold to bring the negative values to 0 without touching the positive ones. It is used for its low power consumption, because it performs a maximum evaluation with 0. In general, the layers rely on this option:

$$f(x) = max(0, x) \tag{2.13}$$

• Softmax: Used for classification, it converts the output value of the layer in scores with a probability distribution by taking the exponential of each output and normalizing these values by dividing using the sum of all the exponentials. In contrast, if all the values in the output layer array of the elements are summed, the sum must be 1:

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \tag{2.14}$$

2.3.2 Convolutional Neural Network (CNN)

A Dense Neural Network has each layer's neurons connected with all the neurons of the subsequent layer. However, there are other neural networks called convolutional, which are more expensive in terms of energy consumption and computationally complex and operate in 2D to apply small filters to scan the input image, which in the case studied would be a spectrogram. It extracts the most prominent features of each area.
In this section, only the essential notions used in this thesis in the CNN field will be introduced.
The architecture of a standard CNN[20] is composed of the structure in **Figure 2.6**:

1. Input Layer - Typically, a CNN receives in input cubes images, with three dimensions (height, width, depth), but in this case, the depth side is not relevant, because the spectrogram is a 1-depth image, requiring only width and height (40x40).
2. Batch Normalization Layer - It applies a normalization to the input to zero mean and unit variance. Stabilizes and accelerates training, helping to accelerate learning. Normalization is performed only during training; instead of model usage, two trainable parameters are training: one scaling for a value γ and one shifting for a value β.

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \cdot \hat{x}_i + \beta \tag{2.15}$$

The parameters are as follows:
• x_i - Input layer
• y_i - Output layer
• μ_B - Mean computation in subset B. B corresponds to batch being the number of samples condensed in an average behavior minimizing errors.
• σ_B - Variance computation in subset B
• ϵ - Small constant error

$$\mu_B = \frac{1}{B}\sum_{i=1}^{B} x_i \quad \sigma_B^2 = \frac{1}{B}\sum_{i=1}^{B}(x_i - \mu_B)^2 \tag{2.16}$$

3. Convolutional Layer - Extracts spatial patterns, from the input layer and computes only on that submatrix the neuron operation of DNN, computing a weighted sum using the kernel, which will be composed by weights, adding the bias, and applying a ReLU activation. It may reduce the dimension with downsampling, or it may be the same, leaving the job to the max-polling layer, instead of adding channels.

$$y_{i,j,k} = \sum_{m=0}^{K-1}\sum_{n=0}^{K-1}\sum_{c=0}^{C-1} \omega_{m,n,c,k} \cdot x_{i+m,j+n,c} + b_k \tag{2.17}$$

K corresponds to kernel size and C corresponds to the number of channels.
4. Max-polling layer - On channels, so to the whole image directly, reduces height and width by a window sliding for a pool size, and among the values takes the maximum one. It provides spatial invariance, reducing small translations and reducing overfitting by downsampling. The overfitting phenomenon is when the model learns the training data too well, including noise and outliers, rather than underlying patterns that generalize the input data. The phenomenon is seeable if the model

performs good training, but fails in validations.

$$y_{i,j,c} = max_{0<m<P}\ max_{o\leq n<P}\ x_{i\cdot s+m,\, j\cdot s+n,c} \tag{2.18}$$

P is the size of the polling window, s is the stride, and max is the maximum value in the pooling region.
5. Fully Connected Layer - To recognize the belong to a particular class, it requires at least one FC layer to perform a classification, and it is like in DNN.

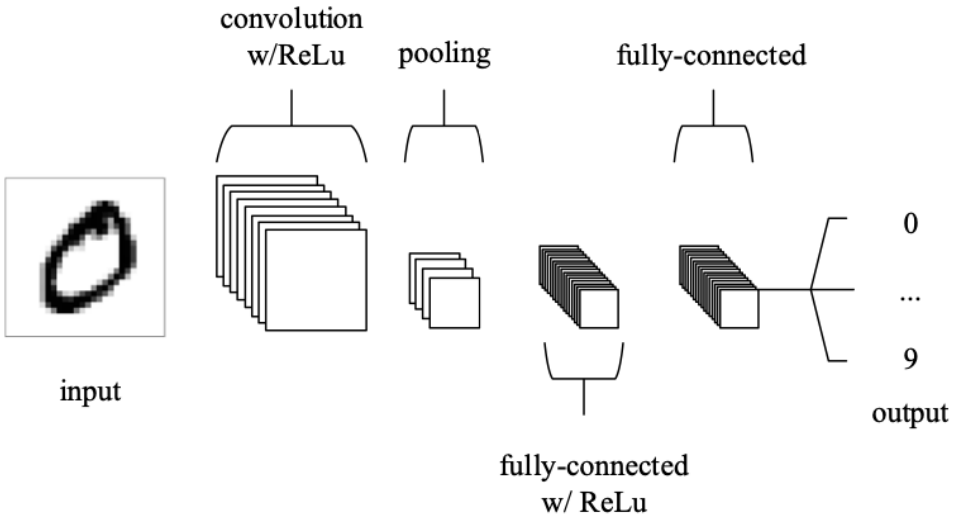


Figure 2.6: Convolutional Neural Network Architecture [4]

2.3.3 Neural Network Parameter Performance

Confusion matrix consists in a visualization method to classify the samples given in input to a neural network. The elements on both rows are columns are the same, but the row corresponds to the result obtained, instead the column to the result expected. In case of binary classifications, the elements can be True Positive (TP), the user expected a correct output and the system validated it, False Negative (FN), the user expected a correct output and the system outputted that it is false, False Positive (FP), the user expected a wrong output and the system outputted a right one, and, ultimately, True Negative (TN), the user expected a false output and the system gave a coherent result. Thanks to this, it is possible to compute various performance metrics:
• Accuracy - Measures the proportion of correct predictions over all predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

• Precision - Measure the right computed instances over the ones that the model given corrected as output:

$$\text{Precision} = \frac{TP}{TP + FP}$$

• Recall - Measure the right computed instances over the ones that the user said were corrected:

$$\text{Recall} = \frac{TP}{TP + FN} = TPR$$

• F1 Score - Harmonic mean of precision and recall, balancing false positives and negatives:

$$F1 = \frac{2 \cdot precision \cdot recall}{precision + recall} = \frac{2TP}{2TP + FP + FN}$$

• Classification Error - Represents the prediction error:

$$\text{Classification Error} = \frac{FP + FN}{TP + TN + FP + FN} = 1 - accuracy$$

• AUC (Area Under Curve) - Evaluates classifier performance across all thresholds via ROC curve:

$$\text{AUC} = \int_0^1 \text{TPR}(FPR)\, d(\text{FPR}) \rightarrow FPR = \frac{FP}{FP + TN}$$

• EER (Equal Error Rate) - The rate at which the false acceptance rate equals the false rejection rate:

$$\text{EER when } FAR(t) = FRR(t) \quad FAR = FPR \quad FRR = 1 - TPR$$

• EER Threshold - The decision threshold value at which EER occurs:

$$t_{\text{EER}} = \arg\min_t |FAR(t) - FRR(t)|$$

• Cosine Similarity - Measures how similar two vectors are:

$$\cos(\theta) = \frac{\vec{x} \cdot \vec{y}}{||\vec{x}|| \cdot ||\vec{y}||}$$

2.4 Speaker Verification Approaches

The purpose of speaker verification is to identify whether a user is part of a database or not. The networks can be trainable with specific user samples, bringing the disadvantage of requiring retraining each time a user is added to the system; threefore during inference and product release this approach is not recommended. Another solution is to choose a one-time training approach, providing a large database of various people. It will generate a feature extraction that, compared with a reference array of equal size, can provide the cosine similarity parameter. For application, the second one is the best but requires more training, a variety of data, and a good amount of samples. Another key difference of the model remains its text dependence. If the model is text-dependent, the reference samples of two different words of the same user will be different ones, requiring more memory space allocation with more words required; instead, a text-independent approach, which is difficult to achieve without retraining, consists of a memory optimization. In this case, the user does not care about saving samples of the words he is saying, but relies only on his data reference. The no-retraining requirement gives feasibility; instead, text dependency will provide more accuracy on single words but will have more memory expenses. It is not the case, but if dealing with a system that does not require user addition in inference, a solution may be relying on a retraining approach and text-independency. This may be possible only if a large dedicated database is provided. There are some relevant solutions, to implement an SV model, however, no one is deployable in a Tiny system, except for a d-vector extractor technique, which relies on finding patterns in voice before the classification process[22].

2.5 Keyword Spotting Approaches

The goal of the algorithm is to train a neural network to recognize whether it belongs or not to a class output independently of the user. The class can be trained on a sample word or a short phrase. To perform the task efficiently on Syntiant NDP101, it is recommended to develop it using the Edge Impulse Framework, which allows a good compression and validation process of the model. The limitations rely on retraining because in performing a text-dependent approach to perform a classification the system should be aware of how a class is composed and characterized. Complex and optimized methods already exist for Embedded Systems, like RNN-based models. This approach uses a recurring neural network[11], which processes a word on the character level and, unlike SV models, the various solutions can be deployed for TinyML devices. Edge Impulse, by default, provides directly a model structure compatible with the MCU, because the NDP101 architecture is very limited, but it would not rely on a complex dataset and it is a more feasible solution, unlike the SV model.

3 KWS and SV Models

3.1 KWS Model

Keyword Spotting (KWS) is a model specialized in audio classification whose objective is to detect the presence of a spoken word or phrase according to an input sample, in this case, the spectrogram extracted by the MFE block. This transformation from raw data to spectrogram provides a time-frequency representation that emphasizes relevant acoustic features. A KWS model is trained to capture a predefined keyword by learning its spectral patterns across a range of samples, which must be differentiated and of big size to ensure generalization across speakers and noise conditions. In this thesis, the KWS model is destined for deployment on the Syntiant NDP101 hardware. Due to the hardware constraints and design recommendations given by Syntiant, a DNN architecture has to be employed with a maximum output of 64 classes[25]. This network has 3 hidden fully connected layers and one output layer, used along with a softmax activation, mapping the feature vector into a probability distribution over defined class labels. The neural network structure strikes a balance between efficiency and classification accuracy, making it suited for real-time applications on embedded devices. The focus of this work is on system verification. It allows the creation of a simple model consisting of one word, basically creating a binary output, that word or not. The chosen keyword for training is Sheila, selected due to its availability within the Google Speech Commands dataset [33]. This offers a good number of audio recordings designed for keyword classification research, including many samples from multiple different speakers with varying environmental conditions. The recordings in the dataset align with the Syntiant NDP101's input specifications because they are sampled at 16kHz and have a one-second duration per each sample. In addition to Sheila's samples, a good dataset should include a variety of background noise sounds, samples of words not present in the dataset, and others similar phonetically, which may look alike from the neural network perspective. To give more samples as possible to the dataset, some audio clips were manually recorded or sourced from a dataset curated by Edge Impulse [15], which is a platform specializing in machine learning for model development and deploying on edge devices. It is important to note that in the dataset provided, there are some Sheila samples, so to avoid any error in splitting, a checking is suggested. The final dataset used consists of 3403 sheila words (almost 56m 43s of audio recording). These samples were divided 80% for training and 20% for evaluation. This division ensures that meaningful representations are learned while it is still providing independent data for validation. To facilitate deployment on NDP101, the model is developed and trained using the Edge Impulse Studio[14], which provides a good and intuitive pipeline for building and optimizing models tailored to the embedded hardware. The processes done by this framework include automatic data preprocessing, training, fine-tuning, and a quantization model conversion to int4, which is compatible with the Syntiant device. This workflow reduces development time, ensuring at the same time that the resulting model fits in the memory. During and after training, confusion matrices are generated for classification performance. These provide insights into which keywords were misclassified, highlighting potential areas of confusion and suggesting model refinement. They are useful in identifying failure cases, such as confusion between similar-sounding words and to avoid this inconvenience, these words were already added to the dataset as others. The DNN model structure of KWS is the one in **Figure 3.1**.

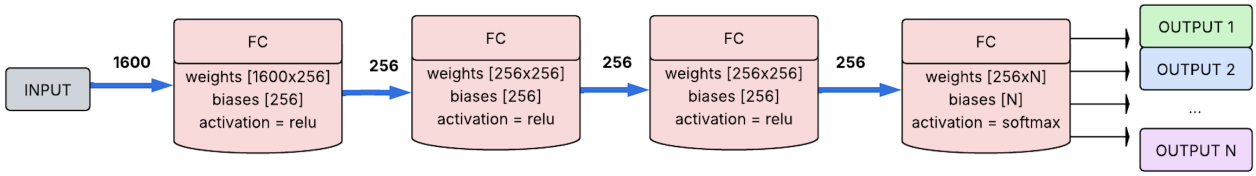


Figure 3.1: Keyword Spotting Model

3.2 SV Model

After creating the KWS model, the objective is to create a text-dependent Speaker Verification model, which requires only one train. This is known as ASV (Adaptive Speaker Verification), which relies on comparing the results of the model (d-vectors) with reference samples stored in the system dataset, which have to be captured during the inference phase. Speaker verification is used to recognize the identity of a user in an on-device learning context. To approach this way, a large amount of data is required at first to train the model, performing a meticulous extraction of d-vectors to recognize patterns in user voice and possibly trying to minimize the required number of samples, maximizing the security recognition. A sample, because of text dependency, will correspond to a word said by one user and should compare only with its similarities.
The deployment of the model requires three caveats:
1. Adapt directly to the device, meaning that a new user should be able to enroll in the SV application by providing voice samples in real time through the target device.
2. The algorithm must operate in a one-class manner and should be able to learn to distinguish between the enrolled user and the others, only using the data from the dataset collected during inference.
3. To fit a TinyML device the considerations should be done in memory allocation depending on the device used, so the model should fit in Flash Memory.
To obtain the desired d-vector we should use a convolutional neural network because we would like to reduce the dimensionality while obtaining significant features. This is achieved with some expensive filters that, while reducing width and height, will generate new channels corresponding to a new feature extracted. This is a way to synthesize the input spectrogram (40 width x 40 height x 1 channel) to a lower dimension.
A valid alternative, in theory, could be i-vectors. It is a characteristic that represents the characteristics of a distributive pattern of frame-level features. The extraction is a reduction in the dimensionality of the GMM supervector[22], allowing an extraction per sentence; instead the d-vector generates a one-hot speaker label on the output, and it is an averaged activation from the last hidden layer of CNN. The advantage of the d-vector is that there is no assumption on the feature's distribution; instead, the i-vector assumes by default a Gaussian distribution.

3.2.1 D-Vector Extractor Creation

The structure of the CNN follows the theoretical one introduced and as input is given the spectrogram is given in output from the MFE block (40x40x1) and as output, the objective is a 256-size long array of relevant features (d-vector). However, it cannot be the model's output because, at first, the model should classify the data given in the input through a fully connected layer. As an input dataset, it is huge with many different speakers, each providing various samples because they come from audiobook recordings. Speakers' classification does not require the text-dependent approach, so the speakers will say many different words, which helps generate the required weights and biases for the convolution layers. The dataset used was from libreSpeech which has data collected per speaker and for this purpose the one with 100 hours of clean speech in the English language sampled with 16kHz was taken, which is the same as Syntiant audio processing[32], corresponding to 6GB memory space to make the training successful in reasonable times.
The first problem is that these samples are not already in 1 second, but are variable, so they have been sampled and parsed through the MFE Block previously introduced. The total spectrograms obtained were 136112 for a total number of classes of 94, almost 38 hours of recording, and a memory occupation of 871MB. To have a balanced dataset, all classes had the same sample number, 1448. At first, they should have been 100, but they had too few samples. A convolution neural should perform three actions during its creation in which the dataset should subdivide its samples:

1. Training - The spectrogram is treated as an image, this dataset is the model's base, and on these the weights are adjusted using backpropagation, like Adam optimizer, and using loss functions like cross-entropy, for classification, or MSE, for regression, to minimize the failure rate. It uses a learning rate, which is a hyperparameter, to determine the step size at each iteration while moving

forward with epochs. An epoch consists of a cycle of training input processing and an evaluation and can be arbitrarily set according to the complexity of the network. In this case, it was set to 700. 2. Validation - This is used to see the result on data other than those in the training set. It is used to adjust the learning rate and batch size in case it starts overfitting or to perform an early stopping if validation loss increases.
3. Testing - It is a dataset to which background noise is added, varying microphone quality, and other modifications to the original input. It is like a validation but to stress out the system and see if it can still work with some fluctuations.

The distribution among these 3 sets is random inside a class, but each one will have an amount in each set. For precision, 70% of the samples will go in training (95278), 15% in validation (20417), and 15% in testing (20417). In training with CNN, a batch size is suggested to avoid fluctuations, and considering that the samples may not be complete words, it performs an attenuation on those cases and the objective is having a good identification of the speaker. Knowing that a larger batch size will provide more accuracy, 32 sizes were chosen, which should be stable with standard learning functions. The setup of the model, as a summary, is an input shape of (32,40,40,1) [batch, width, height, channel] for 2977 inputs (95278/32) and an output shape of 94 classes. The model creation took about 3 hours using a GPU. The proposed architecture[22] corresponds to the one present in **Figure 3.2**:

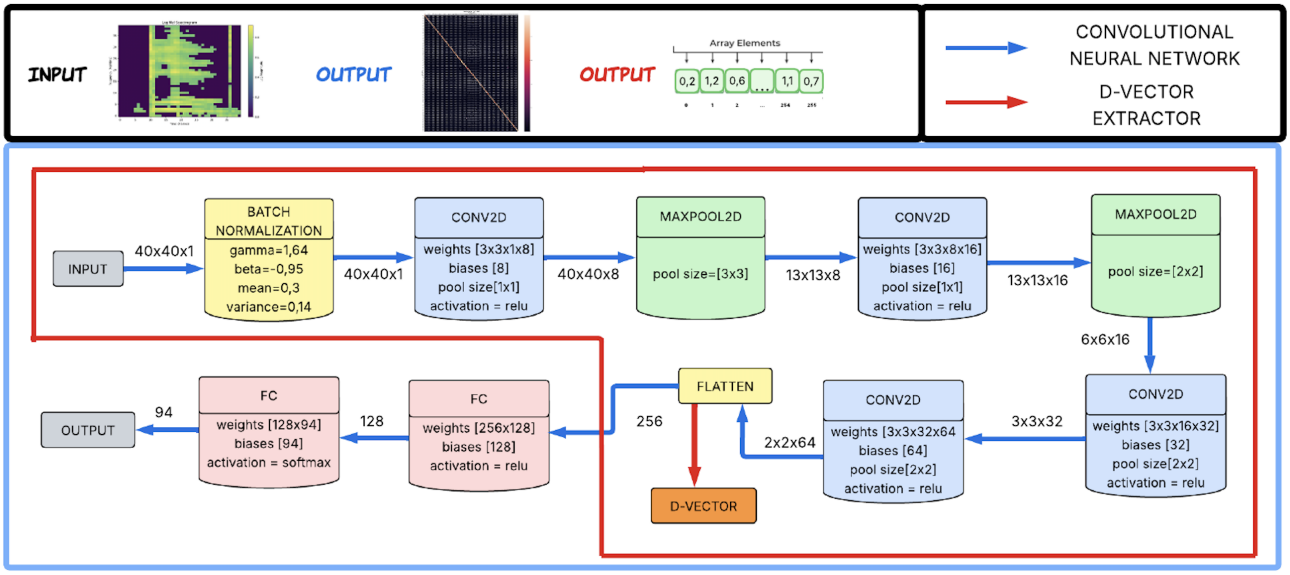


Figure 3.2: Speaker Verification Neural Network and D-vector Extractor

After obtaining the CNN, it was implemented the Fully Connected Layers to catalog and what is expected is even with 700 epochs and 32 batches, to have a good accuracy but a bad loss, because the objective is to find patterns between very different samples. However, before the classification, at the Fully Connected layers' level, there is a generalized array of 256-size that represents the concept of d-vector, a general array that is a compression of the initial spectrogram and with max-pooling leverage relevant features, preserving all its consistency. Knowing this, we could truncate the DNN part of CNN and set the flatten section as the output, and it is possible because the two logic are separated.
The problem that will arise now is how can the accuracy be computed, because before it all depended on classification, but truncating that last part requires another method to capture that value and because this is an ASV model, it will not be trained again. If it had been for a fixed user, for a specific usage, it would have been reasonable, but in this case, it is not the solution. The notion that was previously introduced consisted of cosine similarity. It takes two vectors and gives a percentage output on how much they are similar. The model that has just been created can provide a relevant

reference of the audio sample given in input, and if in a dataset a similar one is provided, even if the spectrogram is not identical with feature compression and filtering they will be very similar. There are some existing solutions on which the model evaluation was performed:
1. Best-matching: During inference, the reference samples have to be recorded in real time. It is recommended, because of this, to accept more than one sample and typically increase the number of samples saved because it is probable to find similar user samples. The best-matching technique saves per word said per user a number of vectors equal to the recommended size. It is important to note that because the d-vector elements are floats, a single reference vector occupies 1KB.
2. Mean Reference: Instead of saving all the reference samples, occupying N KB per user's word enrolled, to save space, the average sample has to be computed. Theoretically, with this method, it would not have an optimal cosine similarity; however, it is a good trade-off in space-saving, and considering the operation on a TinyML the minimizing of storage and memory allocation is important. Some examples of this may be the arithmetical average and the geometrical median.
In the case of Syntiant NDP101, due to the very small space, it is highly recommended to use the mean reference technique to minimize power consumption.

3.2.2 Knowledge Distillation Training

If the solution can be deployed in some TinyML, it is not optimized and not compatible with Syntiant NDP101, because it can only support dense fully connected layer (DNN), but the d-vector extractor is a CNN[18][10]. In machine learning, there is a process called Knowledge Distillation, which is shown in **Figure 3.3**. It adapts two different models using each one with a different architecture but with equal input and output data. To do this, there should be a teacher and a student model. The teacher is a CNN and the student is a DNN that tries to replicate the results of the teacher at feeding of input data. The orientation of the student will be the loss discrepancy, which is 1-cosine similarity. To evaluate the model, the cosine similarity technique can be used for both, but the mean square error is an alternative solution, too. The advantages of this kind of approach are not only for deployment on NDP101, but dense layers have an easier computation than convolutional layers, which require more power consumption. As a consequence, the model would be faster. However, it should require more parameters to obtain similar results, because the 3 hidden layers structure remains fixed. In the end, the model would be less precise and occupy more memory but will be faster and deployable on NDP101.

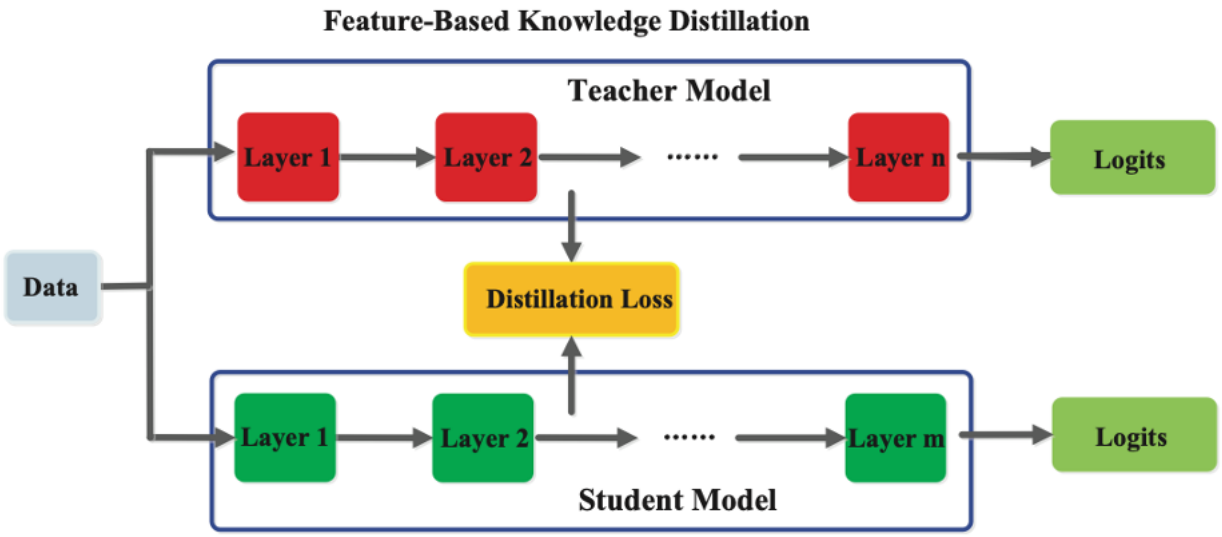


Figure 3.3: Distillation Model from CNN to DNN [10]

The number of neurons per layer has to be chosen by the user, but there are some caveats to consider:
• Syntiant NDP101 supports at most 589.000 parameters, so the neuron choice must be taken into

account
• Downscaling and then upscaling between two layers is not recommended and there should be a reduction in results performances.
• Neurons typically have powers of 2 to optimize space and allow easier weight computation, so in case it is too inefficient, it is recommended to take at least powers of 2 or multiples of 32.

3.2.3 Quantization of SV Model

Unfortunately, the problems do not end here. Syntiant NDP101 not only has a DNN, but requires a parameter quantization, too. The weights should be stored in 4-bit[24] integers, meanwhile, biases in 32-bit integers. Performing a quantization is the most straightforward way, and in the case of transposing it into int-8 it is immediate, thanks to the support provided by Edge Impulse with Post-Quantization Training (PQT)[27], which is the case considering that the distilled model is trained as it is shown in **Figure 3.4**. However, int-4 quantization is not optimized and as of now consists of a fake quantization, such as storing in int-8 bits an int-4.[34] But, it can be seen that PQT int4 weights have really poor performances. Instead, performing a Quantization-Aware-Training (QAT)[28] as of now can achieve decent results, always using fake quantization. The problem is that it is unknown how the Syntiant binary is built and composed, so even with a good packing-unpacking technique, it could be difficult to obtain something from the model. Syntiant NDP101 provides a tool with the SDK to convert a model to binary Syntiant compatible, but it is under NDA. A solution could be using Edge Impulse Framework[17], but it supports only classification and regression models as of now and is not what the d-vector extractor is doing.

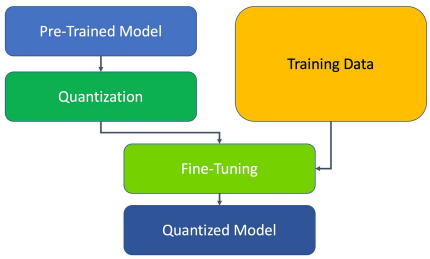


Figure 3.4: Quantization with Tensorflow [27]

However, in theory, to successfully quantize the model, data must perform brute-force quantization and then fine-tuning to adjust values passing database data. An advantage of this is that the model's size stays not only in adjusted weights but reduces the intermediate input and output layers, too, leading to having an output no longer of 1KB, but of 256 bytes. It will save even more space in memory, however, to be sure to convert the input spectrogram in int, the model can be fake-quantize to leave the input and the output as floats adding a quantize and dequantize layer, the ones inside will be int8 and then manually remove from dequantize layer. This will allow maintenance of floating spectrogram logic, but at the same time allowing inference on Syntiant, if the conversion tool was available, and saving memory due to the reduction of 75% per sample. The structure of the quantized model is shown in **Figure 3.5**.

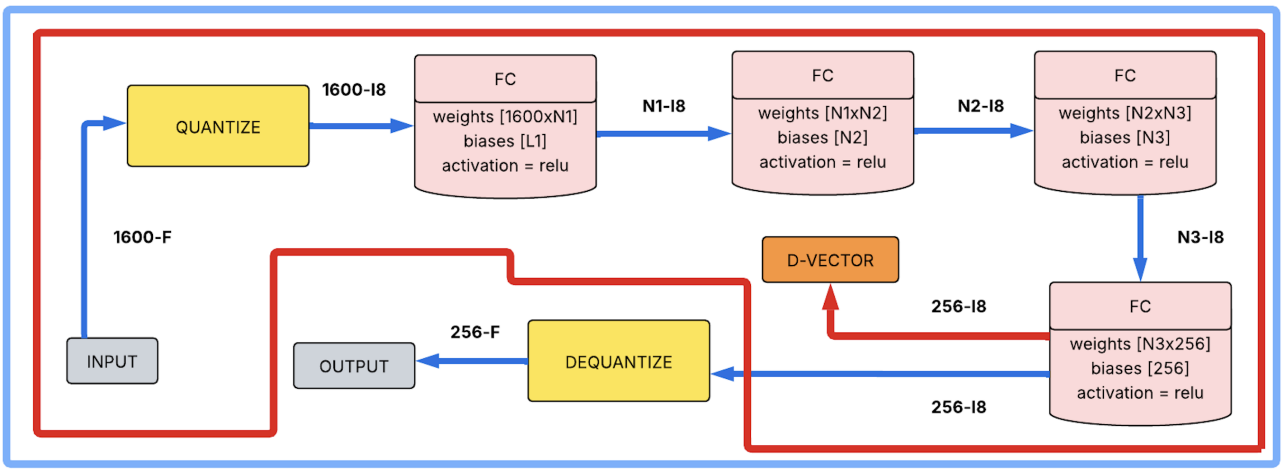


Figure 3.5: Quantized Model Int8

4 SW and HW Implementation

The overall system will work as shown in **Figure 4.1**:
1. The microphones capture the user's voice and save it in the buffer
2. Meanwhile the buffer is being filled in the respective devices, the MFE block is being triggered and its spectrogram output is given in input to KWS segment. It will process the input in Syntiant Pipeline, meanwhile SV performing device will wait for a response.
3. The KWS routine finishes and it communicates the classification output to the SV device via SPI
4. The SV, according to the output, if it is a class, computes the Syntiant pipeline.
5. After extraction of d-vector, the system compares the result obtained with the reference vectors of the corresponding word, limiting unnecessary comparisons.

The possible outputs of the system are:
• No word is recognized, so the SV is not triggered and will not process the spectrogram.
• The sample corresponds to a word, but the user is not enrolled for that specific class.
• The sample corresponds to a word and the user is enrolled for that specific class.
According to these different outputs, the programmer will be free to perform a connected action.
This thesis, at first, was developed considering the system implementation on two Syntiant NDP101 devices with the objective of creating a multi-model system. Ultimately, because an NDA was not possible to deploy, the overall idea will be presented as if access to the SDK was granted. A multi-model system consists of having a uniform signal processing with an appropriate reshape and MFE block processing by an NDP101 and only then the result will be parsed to the other device to perform a different model action. During the dissertation of the methodology used in the software pipeline, two different approaches would have been followed:

• Simulation on a computer (Software Approach) - Created to verify system pipeline correctness before deployment. It does not use any hardware component, except the microphone integrated into the computer. Shows the behavior using pure C code and saving the models in header.
• Inference on MCU (Software + Hardware Approach) - Actually application deployment, which requires handle of hardware components. In this case, the models are uploaded in binary files, but each one is on a different MCU, not like the simulation, which was all accessible by the same compiler. The hardware and communication (SPI) had to be managed, but the other phases are equal to the ones edited in C in the simulation.

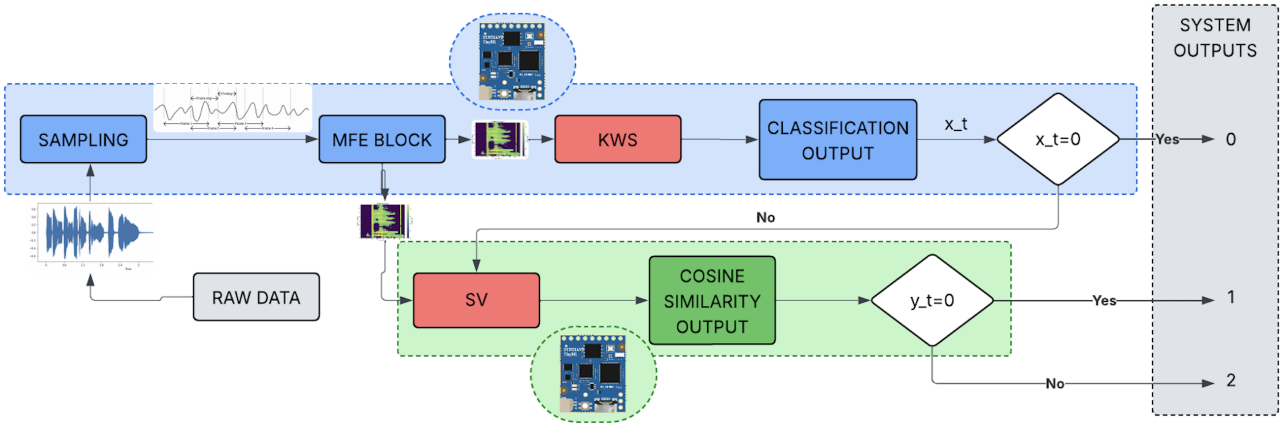


Figure 4.1: Software Pipeline

4.1 Software Pipeline

The system was developed in simulation and in validation using a simulation software approach, instead in inference only the code logic inside the single NDP101 was developed. In this section, we will talk about:
• Signal Capture (Simulation)
• MFE Block Generation and Processing (Simulation)
• Model processing (Simulation)
• Output Elaboration (Simulation+Inference)
• Enrollment (Simulation+Inference)

4.1.1 Signal Capture

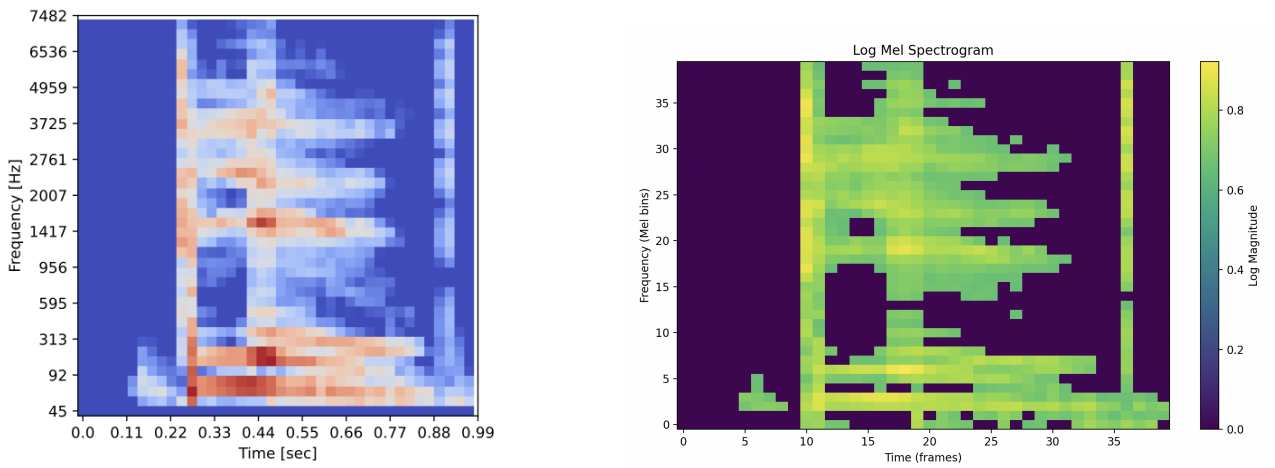
The model requires real sampled raw data to work. If in Syntiant NDP101 there is a dual microphone integrated that computes the ADC conversion, three different modes were created on software to handle the code, according to the needs:

1. Live Sampling Mode (Mode 0) - The performing of live sampling from real input was performed to do fast debug, without the need to upload and create different files each time. The idea is to be a one-shot code, so when launched it activates the audio capture for one second and then uses that input audio as if it were the raw data inputted by the microphone. To do it, an external library called PortAudio[7], which manages the input audio flow, starts it, stops it, and then calls a callback function to trigger the rest of the system.
2. Files with .wav extension (Mode 1) - To perform validation, like a confusion matrix, multiple files should be parsed, which could not be done with live sampling. The idea was to create this mode that takes in input a .wav file that, with appropriate manipulation, would give the same output as PortAudio, but it can be reiterated. More precisely, using a bash file can be given in input a folder and the program will be called several times equal to the .wav files present in it. However, at the same time, it can be parsed as a single file. This is the primary technique used in the validation of graph performance model.
3. Elaboration using data stored in a header (Mode 2) - The most primitive technique among the three proposed, but was useful in code creation and verification, and it consists of copying and pasting the digital values into a header file.

The properties of the audio computation were adapted from Syntiant, and for easiness sampled one second and then truncated the last 0.032 seconds, to have all samples long 0.968 seconds, which with a frequency of 16kHz, corresponds to 15488 digital elements.

4.1.2 MFE Block Generation and Processing

Previously, the theoretical and mathematical computation of the Syntiant block, which is similar to the MFE Block, was introduced with passages consisting of preemphasis, framing, mel-filterbanks, and noise floor. The code follows the mathematical logic, however, to support Fast Fourier Transformation, to be sure in using a functional and optimized code was used FFTW library, which has already implemented functions for Fast Fourier Transformation[8]. To verify the correct computation and application of the theoretical concepts, I relied on the Edge Impulse code[16]. This is a site on which deployment can be easily performed and provides a tool to convert input raw data to spectrogram features compatible with Syntiant NDP101. However, the Syntiant block code is not fully available and requires a trial-and-error implementation. To be sure that the code was corrected, a viewing validation was performed because Edge Impulse gives as output a visual spectrogram, so a Python code was implemented and called via a bash file after program computation that prints out the spectrogram features obtained, adjusting values until the two images matched or were very similar, as in **Figure 4.2a** and **Figure 4.2b**. The pseudocode for the logic is present in **Algorithm 1**.



(a) Edge Impulse Spectrogram

(b) Custom Code Spectrogram

Figure 4.2: Comparison of Spectrograms

Algorithm 1: Spectrogram Computation Pipeline

Input: Raw audio signal `audio[]` with `num_samples` samples
Output: Log-Mel spectrogram `log_mel_spectrogram[]`

Normalize and apply pre-emphasis:;
  for $i \leftarrow 0$ **to** $num\_samples - 1$ **do**
    $norm[i] \leftarrow audio[i]/32768.0$;
  $pre\_emphasis\_array[0] \leftarrow norm[0]$;
  for $i \leftarrow 1$ **to** $num\_samples - 1$ **do**
    $pre\_emphasis\_array[i] \leftarrow norm[i] - \texttt{COEFFICIENT} \times norm[i-1]$;
Slice into overlapping frames and apply Hamming window:;
  foreach *frame* f **do**
    $fft\_in \leftarrow$ frame of size `FRAME_SIZE` from $pre\_emphasis\_array$;
    for $i \leftarrow 0$ **to** $FRAME\_SIZE - 1$ **do**
      $fft\_in[i] \leftarrow fft\_in[i] \times (0.54 - 0.46 \times \cos(2\pi i/(FRAME\_SIZE - 1)))$;
  Compute FFT and magnitude spectrum:;
    $fft\_out \leftarrow \texttt{FFT}(fft\_in)$;
    for $b \leftarrow 0$ **to** $NUM\_BINS - 1$ **do**
      $spectrogram[f][b] \leftarrow \sqrt{fft\_out[b].r^2 + fft\_out[b].i^2}$;
Construct Mel filterbank:;
  Generate $FILTER\_NUMBER + 2$ evenly spaced Mel points between MIN_FREQ and MAX_FREQ;
  Convert Mel points to Hz and bin indices;
  foreach *filter* j **do**
    Construct triangular filter between left, center, and right bins;
    Assign weights to $mel\_filterbank[j][\,]$;
Apply Mel filters and compute log-Mel spectrogram:;
  foreach *frame* f **do**
    foreach *filter* j **do**
      $sum \leftarrow \sum_k spectrogram[f][k] \times mel\_filterbank[j][k]$;
      $log\_mel\_spectrogram[f][j] \leftarrow 10 \times \log_{10}(sum + \varepsilon)$;
Apply noise floor and quantization:;
  foreach *value in* $log\_mel\_spectrogram$ **do**
    Normalize with respect to $NOISE\_FLOOR$;
    Quantize to range $[0, 255]$, then normalize to $[0, 1]$;
    Zero out values < 0.65;
return $log\_mel\_spectrogram$

4.1.3 Models Processing

The software pipeline first inferences the KWS model and, according to the result, if higher than 0, triggers the SV one. In simulation, they are handled with the components of the neural network explicitly shown. To weights and biases shall be declared to recall. At the same time, the input and output allocation of each layer should be allocated with respect to the functions that were created, the reasoning of the fully connected layer process, and the corresponding activation functions. The Syntiant model elaboration code for both the KWS and SV models follows the **Algorithm 2**[1], following the theoretical concepts.

Algorithm 2: Neural Network Inference Example

Input: Input vector $x \in \mathbb{R}^{\texttt{INPUT\_SIZE}}$
Output: Predicted class index y
Initialize:
    Create hidden layer buffers: $fc_1 \in \mathbb{R}^{H_1},\ fc_2 \in \mathbb{R}^{H_2},\ fc_3 \in \mathbb{R}^{H_3},\ output \in \mathbb{R}^{C}$;
Feedforward pass:
    $fc_1 \leftarrow \texttt{ReLU}(W_1 x + b_1)$;
    $fc_2 \leftarrow \texttt{ReLU}(W_2 fc_1 + b_2)$;
    $fc_3 \leftarrow \texttt{ReLU}(W_3 fc_2 + b_3)$;
    $output \leftarrow \texttt{ReLU}(W_4 fc_3 + b_4)$;
Softmax normalization (if classification needs):
    $\texttt{softmax}(output)$;
Sort and classify:
    Sort *output* in descending order, track original indices ;
    Print top-C class probabilities and names ;
return *index of class with highest score*

4.1.4 Output Processing

After the model elaborates its output the result should take several directions, for the KWS model classification method consists simply of using the output class to perform the desired action, like triggering the SV model, and can be simply done with an if-case in simulation and with SPI sending in inference, but different case is for SV model. To evaluate the result, a cosine similarity approach is used, consisting of comparing two different reference d-vectors to find a percentage of how much they are similar. Two different techniques of using reference features were explored in the previous chapter, but to compute the similarity to a mathematical formula consisting of the scalar product of the two vectors long N, divided by the product of their Euclidean norms:

$$cosine\_similarity(x, y) = \frac{x \cdot y}{|x| \cdot |y|} = \frac{\sum_{i=0}^{N} x_i y_i}{\sqrt{\sum_{i=0}^{N} x_i^2} \cdot \sqrt{\sum_{i=0}^{N} y_i^2}} \tag{4.1}$$

This will be done among reference samples using the techniques of best-matching and mean-reference, previously introduced. So, after the comparison with the d-vector stored in the dataset according to the system threshold, the result will be binary: 0, if a d-vector similar to the one given in input in the dataset was not found, and 1 if yes.
The structure of the dataset is first cataloged by words, then by user, and finally, per user's sample, if using the benchmatching technique. This system simplifies the overall computation, reducing the number of reference d-vectors to compare with the input one, simply giving the allocation in memory corresponding to the word selected and all the samples will be in a contiguous memory allocation to use spatial property, accessing only to the subsequent element, and uses information that it should receive anyway. Another consideration is that if a sample receives a similarity above the threshold, the system exists directly, and to output 0 it should parse all samples associated with the word given by KWS. The behavior is like the one in **Figure 4.3**

[1]Note that this is a general processing of a model because in the case shown KWS performs a softmax needed for classification, but it is not mandatory, for example SV performs only a ReLU. In the algorithm, the KWS is shown but has to be adapted to the needs of the user's model

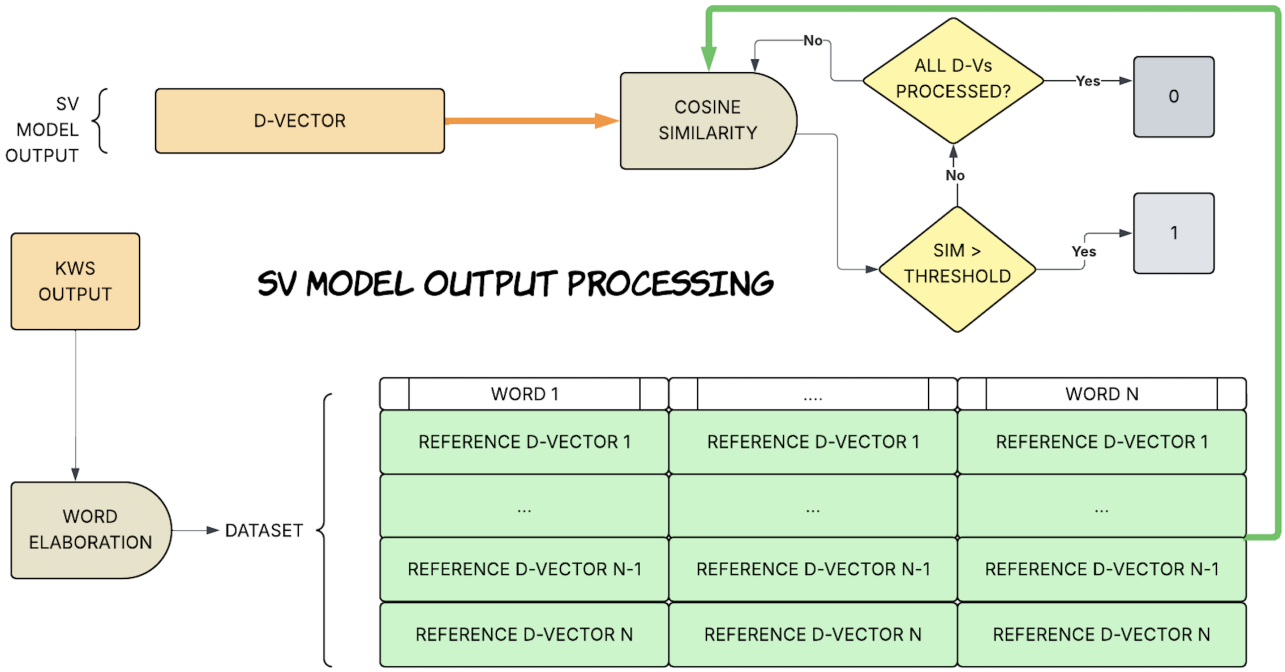


Figure 4.3: D-Vector Processing according to Database

4.1.5 Enrollment phase

On inference, the complete application of this method is restricted due to the impossibility of quantizing the model to 4-bit precision and manipulating SPI interfaces. This limitation is primarily due to the nondisclosure agreement (NDA) surrounding the Syntiant NDP101, which prevents direct control over its internal functions. However, the Speaker Verification (SV) model was trained on a large and diverse dataset to generate a distinct d-vector representation for each user, enabling a one-time training process. In this scenario, the user needs to enroll by providing several voice samples. This number should be balanced: not too large to avoid excessive memory usage, but sufficient to maintain good recognition accuracy. The idea is to reuse the existing pipeline by replacing only the SV model inference stage. When enrolling, the user specifies the desired keyword, after which the system begins recording. However, it only saves samples that trigger the Keyword Spotting (KWS) model. Once the required number of valid samples is collected, the d-vectors are stored sequentially in memory by determining the current end of the dataset and appending the new d-vectors accordingly. The memory structure for the dataset is designed such that each keyword is allocated a fixed portion of memory. In addition, a separate structure tracks the number of samples collected per keyword. Since each d-vector has a fixed length, it becomes straightforward to calculate the memory offset for appending new vectors. In the simulation, this dataset resides in a header file for simplicity, whereas in the actual inference phase, only the logic has been implemented and the full system integration has not been completed due to the limitations in connecting with the KWS system on hardware. Ideally, the dataset would be stored directly in internal SRAM, taking into account its typical 256-byte size constraint. However, some microcontrollers (MCUs) support external SD card storage, which could be used as an alternative. Using a memory mapping technique to manage allocation could be effective even with larger memory sizes and is unlikely to significantly impact energy consumption. Nevertheless, internal storage is generally preferable and should be tailored based on the remaining available memory after code and global variable allocations. A best-matching approach, where multiple samples per user are stored and compared, typically yields higher accuracy, but consumes more memory. Alternatively, averaging the d-vectors into a single representative vector per user results in lower precision but significantly reduces memory usage as it eliminates the need for storing multiple individual vectors.

4.2 Hardware Pipeline

Viewing the system from a hardware perspective, it will be made up of 2 Syntiant NDP101, one being the master device and the other the slave. The MCUs are both always on-devices, so they are continuously capturing audio, but the device that is handling KWS will always process all its software pipeline, instead SV one requires a trigger because of it being a slave. Going on or not should be setup with a SPI communication, which on Syntiant is provided by a internal SDK tool, allowing to setup 4 out of the 5 GPIO ports. The benefits of using an SPI communication stay in no interruptions, during the data transfer, so it would be a continuous stream, allowing the master device to perform synchronous and serial communication with the device. Another reason why this is preferable is the scalability, because knowing that Syntiant NDP101 could not hold much memory, to avoid more energy consumption with an external memory, other Syntiant NDP101 may be used for different words and create an hardware logic to redirect the sample to the correct microcontroller. The ports' settings to allow for this communication are:

- MOSI (Master Output/Slave Input) - line that sends data/trigger to the slave
- MISO (Master Input/Slave Output) - line that slave uses to send data to the master
- SCLK (Clock) - line for the clock signal
- SS/CS (Slave Select) - line to select which slave to send data to, allowing for the scalability

The setup in practice is like the one in **Figure 4.4**:

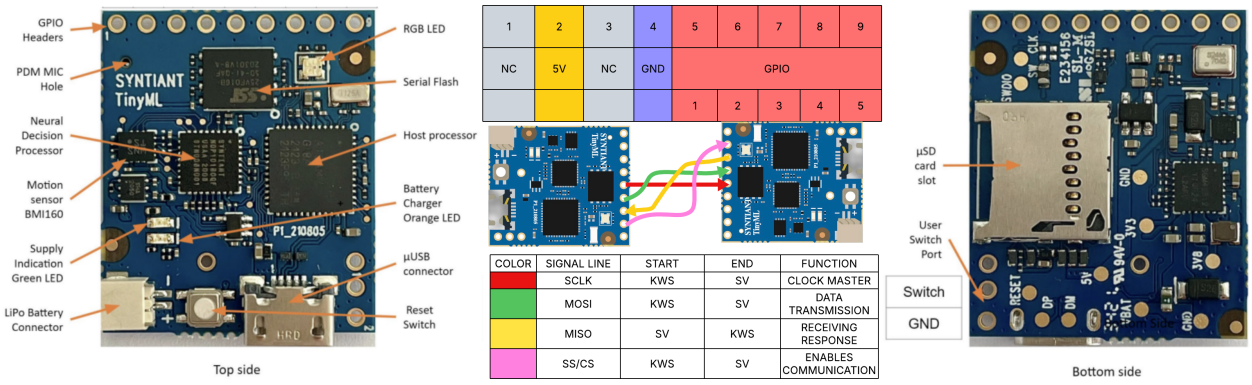


Figure 4.4: System Hardware Setup

The pseudocode of each MCU consists in a setup function, called only once when the devices are turned on or after a reset and another that is continuously running and it is instead the core of the implementation. The NDP101 is composed of sub-elements like the audio buffer that are not programmable, the only functions that can have some code uploaded can be the Flash Memory, to upload the trained AI model and the SRAM memory for the output management and other hardware component configuration. The standard initialization of the microcontroller consists in loading the generated model, setting up the peripherals and the modalities, like low-power mode, to ensure the minimum energy consumption, so only when the microphone will recognize a distinguishable noise will it run the model. This is done via the WFI implementation (Wait For Interrupt), which handles in an efficient way the computation. When a relevant sound is detected, the peripherals are enabled again, and the interrupt NDP_ISR is called. When the interrupt is triggered, the audio is automatically converted via the MFE block and gives the result as input to the model. If the model configuration and operations are directly in the binary file, the only thing that needs to be programmable is the output processing, distinguish the classification with a softmax activation function in KWS model case and a more composed logic with the dataset initialization, population, and comparison of d-vectors via cosine similarity introduced before. The code to perform both models on Syntiant has been written; however, was verified only for KWS, because of the inability of generating a model 4-int weight Syntiant compatible quantization for the SV model.

5 Results Obtained

5.1 Test Models

This chapter shows the performances of the algorithms previously discussed, focusing on a standard version developed on Edge Impulse[14], two d-vector versions, one emulating an existing solution[22] (d-vector extractor size 256) and another solution with size 128, that could be adapted to the system, to reduce the required dimension in the dataset storage, and five distilled models (one from the 128 version and four from the 256 version). The reason why various versions of the 256 d-vector size were generated is because of lack of fitting. To be deployed on Syntiant after distillation, the models have to be quantized in 4-int weights. Knowing that the model can host at most 589.000 4-int parameters, the solution for the 128 version is straightforward in converting to a consistent dense model; in contrast, the 256 version is harder to adapt to these tiny dimensions. To determine if a model can be hosted in a Syntiant NDP101, knowing that the dense layers are 3 intermediate and one output. The formula for computing the model size of a dense neural network is the following:

$$model\_size = \sum_{i=1}^{N}((in\_size_i + 1) * out\_size_i) \quad condition: \ model\_size < 294.500\, bytes$$

Instead, to compute the one of a convolutional neural network should be used this one because of multidimensionality, considering a general filter shape like (width, height, channels):

$$model\_size = \sum_{i=1}^{N}((in\_channels * f\_width * f\_height + 1) * out\_channels)$$

In the following, there is **Table 5.1** summarizing which model will be deployable on Syntiant: Note

Model Name	KWS	SV128	SV256	SVD128	SVDU256	SVD192	SVD240	SVD2256
Type	Dense	Conv	Conv	Dense	Dense	Dense	Dense	Dense
Origin	-	-	-	SV128	SV256	SV256	SV256	SV256
Input	1600	(40x40x1)	(40x40x1)	1600	1600	1600	1600	1600
Layer 1	256	(13x13x8)	(13x13x8)	256	256	192	240	256
Layer 2	256	(6x6x16)	(6x6x16)	256	256	192	240	256
Layer 3	256	(3x3x32)	(3x3x32)	256	128	192	240	256
Layer 4	-	(2x2x64)	-	-	-	-	-	-
Layer Out	2	128	256	128	256	256	256	256
Total (KB)	2117	383,6	95,2	2243,5	2115,5	1683,25	2193,8	2372
Deployable	YES	NO	NO	YES	YES	YES	YES	NO

Table 5.1: Model Dimension Details

that the width and height of the convolution formula are used for the filter size; instead, in the table above, there are the input and the output of the steps. The filter is always (3x3xchannels), with channels instead of the ones present in the datasheet. The condition for a model to be deployable in float configuration, before quantization, is that it should be smaller than 2300 KB and it should be a dense model. The choice of the distillation models having as father 256-size version are, respectively:
• U256 - consisting of an unbalanced distribution of neurons, having a layer downscaling to 218 from the standard of 256 to fit inside the Flash memory.
• 192 - considering that model neurons should typically have a power of two, considering that 128 neurons in all layers is too restrictive to mimic the father's behavior, a general multiple of two with a reasonable value that fits inside the device was chosen.

• 240 - the optimal multiple of 2 that could fit inside the device, having all the intermediate layers with equal neurons and the output of 256.
• 256 - this model does not fit, but was used to compare the behavior with another model with 256 neurons in intermediate layers and 128 d-vector as output
If the KWS performs a classification having a direct output, SV performs a cosine similarity with d-vectors saved as references. This is done to view various perspectives on which version of the model is better considering performance and size perspectives. The methodologies of d-vector comparison used are 3:
1. Best matching - takes the highest cosine similarity value when comparing each reference vector with the vector obtained as a result of the Speaker Verification model. It requires that all reference vectors be saved in memory.
2. Mean d-vector - the mean d-vector is computed performing the average among each d-vector and then using this obtained vector is performed a cosine similarity and the score obtained will be the result. In this case, only the mean d-vector should save more space.
3. Geometrical Median - it is an optimized version of mean computation. Implements Weiszfeld's algorithm[19], which is an iterative method to compute the geometric median of a set of points present in an Euclidean space. It is more computationally expensive because it first requires a mean computation. Then, some weights computation to minimize the distance between the points, to obtain an optimized d-vector. Weiszfeld's algorithm corresponds to **Algorithm 3** pseudocode.

Algorithm 3: Geometric Median via Weiszfeld's Algorithm

Input: Set of n vectors $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\} \subset \mathbb{R}^d$
Output: Approximate geometric median $\mathbf{z} \in \mathbb{R}^d$
Initialize: Set initial estimate $\mathbf{z}^{(0)} \leftarrow \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i$;
for $t \leftarrow 0$ **to** $T_{\max}$ **do**
    Set $w_{\text{sum}} \leftarrow 0$;
    Set $\mathbf{z}_{\text{new}} \leftarrow \mathbf{0} \in \mathbb{R}^d$;
    for $i \leftarrow 1$ **to** n **do**
        Compute $d_i \leftarrow \|\mathbf{x}_i - \mathbf{z}^{(t)}\|$;
        if $d_i < \varepsilon$ **then**
            continue ; `// Skip or optionally snap to` $\mathbf{x}_i$
        Compute weight $w_i \leftarrow \frac{1}{d_i}$;
        Update weighted sum: $\mathbf{z}_{\text{new}} \leftarrow \mathbf{z}_{\text{new}} + w_i \cdot \mathbf{x}_i$;
        Update weight sum: $w_{\text{sum}} \leftarrow w_{\text{sum}} + w_i$;
    if $w_{sum} = 0$ **then**
        break ; `// All points are too close; cannot proceed`
    Normalize: $\mathbf{z}^{(t+1)} \leftarrow \frac{\mathbf{z}_{\text{new}}}{w_{\text{sum}}}$;
    if $\|\mathbf{z}^{(t+1)} - \mathbf{z}^{(t)}\| < \varepsilon$ **then**
        break ; `// Converged`
return $\mathbf{z}^{(t+1)}$

For each of these methods, the number of reference d-vectors used was 1, 8, 16 and 64; it is relevant to note that one result should be the same in all 3 modes. This reference dataset with size of 64 KB on a SRAM of 128 KB. The components of d-vector changes, because the default size-type model is float, but Syntiant quantizes it in 4-int weights, but the performances could not be verified because of the NDA, so were used in float format.
Because of being a TinySV with an objective of using internal memory for dataset, too, it is important to consider the size, allowing evaluation on how many words and consequently how many users can be inserted in the logic depending on the usage. They are in inverse proportion with $num\,of\,users \cdot words\,categories = total\,num\,fitable\,on\,Syntiant$ and the total number in Syntiant varies according to the method used and the number of references. In the case of bestmatching technique, having a higher number of references leads to having less words, because a word representation will require more space. To map out the dataset dimension of each model, there is **Table 5.2**

Type	D-vector = 128					
Aggregation	Best				Mean	Geom_Median
N° Refs	1	8	16	64	All	All
Size of Word (B)	512	4096	8192	32768	512	512
Quant (B)	64	512	1024	4096	64	64
Words Float	128	16	8	2	128	128
Words 4-int	1024	128	64	16	1024	1024
Type	D-vector = 256					
Aggregation	Best				Mean	Geom_Median
N° Refs	1	8	16	64	All	All
Size of Word (B)	1024	8192	16384	65536	1024	1024
Quant (B)	128	1024	2048	8192	128	128
Words Float	64	8	4	1	64	64
Words 4-int	512	64	32	8	512	512

Table 5.2: Dataset dimensional configuration based on aggregation type and number of references

5.2 Testing Dataset and Performance Metrics

The various models were tested with a composed dataset containing one-second audio samples of my own voice saying a train word (Sheila), the voice of other users saying the same word, but different from the ones used during training[32] and some samples of total different words and some that may sound like the chosen word. In the KWS model, the first two cases will output TP and FN, meanwhile the third category will output FP and TN. Instead, for SV models, only the words that were considered TP would be processed; however, considering that some reference d-vector had to be chosen randomly, an algorithm threshold was created. That one, in case the similarity was above 0.99, the sample has to be truncated. Because of this limitation in Convolutional Models, there are fewer samples, because they were eliminated to not ruin the results; instead, with distillation a loss in cosine similarity is predicted, because it is a try to mimic the convolution behavior with simple multiplication and addition, so even the reference samples passed in the neural network will not be processed as the same ones.

Quantitatively speaking, the test dataset is made up of the following:

• 485 Sheila Owner Samples (registered manual and split into one-second samples using Edge Impulse[1])

• 2022 Sheila Others Samples (taken by the Google Command Dataset[33] corresponding to the Sheila word)

• 474 Similar Sheila Words (taken by Google Command Dataset[33] not corresponding to the Sheila word)

This leads to a total of 2981 samples used for KWS; instead, for SV, to be sure that only true Sheila words were passed, were accepted only with a probability of being Sheila higher than 0.8, so the TP samples of KWS which are higher than that value.

In addition to size metrics, there are some performance metrics previously introduced in **Chapter 2** that will be used in evaluating the performance of each model. The threshold on which a value is determined to be true or not depends on the EER Threshold, which is the best value in which the FPR and the FNR are as close as possible. On this threshold, were computed:

• Accuracy: Rate of results corresponding to expectations over total elements (Higher better)

• Classification Loss: Rate of results not corresponding with expectations over total elements (Lower better)

• Precision: Rate of true expectations over all samples which result is true (High better)

• Recall: Rate of true results over all samples expected to be true (High better)

• F1 score: An estimated score to evaluate performance considering precision and recall (High better)
• EER: Is the highest value among the false acceptance rate and the false rejection rate, even though they should be similar, because the threshold is equal to the ERR point (Low better)
• AUC: Is the area under the curve that in a binary classification that if around 0.5 indicates random guesses, instead higher values represent more deterministic results (High better)

5.3 Experiment performances

5.3.1 KWS

Starting with the analysis of the results obtained, the first model to explore will be KWS. The classification is as known binary, but according to Syntiant, the maximum trainable keyword spotting words are 64.[24]. The computation of the performance calculation such as AUC and ERR was in a range of 0.5 and 1 thresholds. According to the results obtained, the threshold is around 0.7 with 90% of true positive out of 2507, expected true samples, and 90,11% of true negative out of 474 samples. The model overall performs well with a low Equal Error Rate and a decent AUC value, compensating with good accuracy, optimal precision, and good recall, leading to the following statistics:

Configuration	Data Analysis	Performance	Quality
Model: KWS **Threshold:** 0.7 **Total Samples:** 2981	**Confusion Matrix:** TP: 2259, FN: 248 FP: 48, TN: 426 **Distribution:** Pos: 2507 (84.1%) Neg: 474 (15.9%)	**Classification:** Accuracy: 90.1% Precision: 97.9% Recall: 90.1% F1: 93.9%	**Quality:** EER: 0.101 AUC: 0.885

5.3.2 Speaker Classification of CNN Model

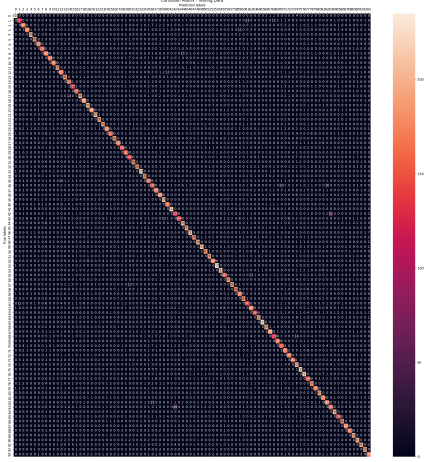


Figure 5.1: Confusion Matrix

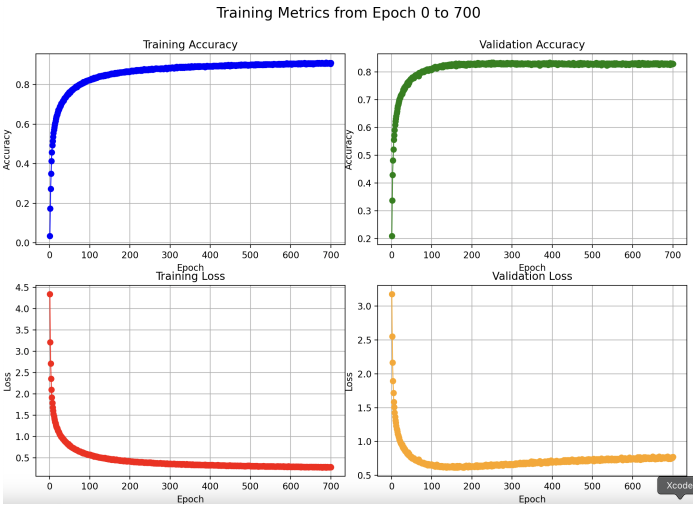


Figure 5.2: CNN Training Details

If KWS is only a single model that can be optimized and managed over time, the purpose of SV is to be a one-train model. Because of this, it is really important to determine the best configuration possible for a deployment. As a starting point, it is interesting to analyze the confusion matrix obtained by the classification of the original convolutional model before truncation. The training was performed with 700 epochs using as accuracy parameters the correspondence of the samples results with the expectations; instead, the loss was computed with two callbacks:
1. ReduceLROnPlateau: In case the training loss does not improve (decrease), the learning rate is reduced by a factor. In this case, a patience of 10 and a factor of 0.5 were applied.

2. EarlyStopping: In case the training loss does not improve, the training is stopped earlier. In this case, it was set to a patience of 20, but most importantly this allows one to restore the best weights from the epoch with the lowest training loss, to get the values, before the model overfits. The overfitting occurs when a model learns data too well, including outliers or random fluctuations, and performs poorly with unseen data. As a result, an accuracy around 83% is obtained; meanwhile, there is a 61% loss. The reason why there is still a high loss is because of the presence of only an intermediate layer, other than the output one, dense layer's talking. This leads to less precision because mapping out 94 users may be difficult in a classification; however, all of this was done only to give the d-vector, to see how accurate it may be using a classification.

5.3.3 SV Convolutional Models

The two d-vector extractors obtained truncating the classification part are of 128-size and 256-size. The advantages of using the 128's one consists in size of the reference d-vectors, which in every case will be the half of 256's and could host the double of the words and it should be performing slightly better with the averaging methods, because it has fewer parameters. However, it will be less precise than 256's solution. These are what in theory is expected, but to verify the correctness in practise the two were viewed from two perspectives, one a general purpose usage, in which it is not important to have an absolute precision, but the objective is using the threshold which gives the maximum true positive rate and the minimum false positive rate, instead the other view is a security one, with authentication application, having a precision equal to 1. The complete results table are in **Table A.1** for the first case and in **Table A.2** for the other. In the comparison between the two models, an outperformance of 256 models over 128 one is seeable. Starting with the optimal EER, the analysis shows that the output size is a crucial factor to obtain better performance, increasing the representational capacity. Each method has the following characteristics:

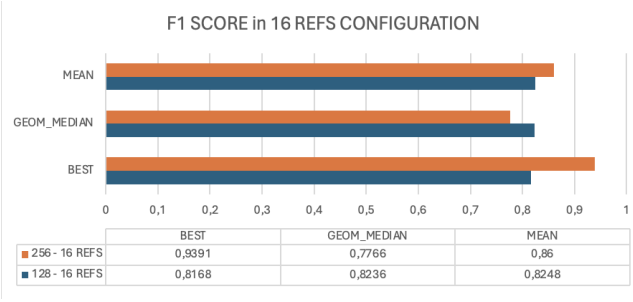
• BEST - show strong performance with an increasing number of references.

• GEOM_MEDIAN - it trails behind BEST, performing closely when the reference number is high, and it is a good compromise for robustness and simplicity, it is better with the 128 model, because of lower output element size.

• MEAN - It shows trends similar to GEOM_MEDIAN with 128 model and slightly better ones with 256.

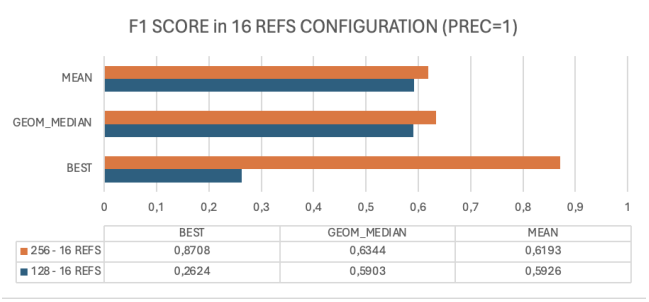
Across all methods, with an increasing reference number, it improves recall, precision and F1 score, and, at the same time, EER is reduced, leading to a better verification reliability. An interesting effect is that both models gain saturate beyond 16, with a slightly improved performance at 64. Considering the thesis objective, it is important that the model performs as best as possible, in the limit that it is user-friendly. In fact, registering 16 references to the system is more acceptable than 64. Below 16 the configurations struggle, showing poop performances. Even if the BEST approach shows better results than the others, it will occupy more memory, so there should be a balance between model accuracy and the size of a word saved on the MCU memory. The data of the models using 16 references are shown in **Figure 5.3**. Using these models it is recommendable in case there is space available to use BEST method with 256 configuration and use MEAN to have lower accuracy and precision, however, it is not recommendable to use GEOM_MEDIAN, because it requires more computation than mean for lower performance, so in this case, probably because there were not many outliers, it has to be avoided. Considering the 16 reference configurations, other than GEOM_MEDIAN performance in general in which 128 outperforms 256, it should be noted that 128 has in the MEAN method a better recall of about 6,41% and an EER 3,61% lower than the other. Instead, talking about no false positive tolerance analysis, because there is a try in maximizing the precision, parameters like Recall and EER, and as a consequence F1 Score, will be worse. It is seeable in **Table A.2**, that the 256-sized model significantly outperforms the 128-sized model, especially at higher reference numbers. It is interesting to note that unlike the EER configuration, the F1 score does not always increase, especially for the BEST method in 256-sized model, due to a drop in precision at higher NUM_REFS. If BEST with 16 references is an optimal solution, because it maintains the values on track, even if it has to be accepted, it is a lower recall, but still acceptable. Instead, the other methods (GEOM_MEDIAN and MEAN) are in any case not usable, because of recall lower than 50%. In this case, to guarantee the model security it is required to sacrifice the size in favor of the optimization and there is no best

method than 256-sized model with BEST method and 16 references. The results on 16 references of this no-tolerance false positive are shown in **Figure 5.4**.



	BEST	GEOM_MEDIAN	MEAN
256 - 16 REFS	0,9391	0,7766	0,86
128 - 16 REFS	0,8168	0,8236	0,8248

Figure 5.3: F1 Score with 16 References (<EER)

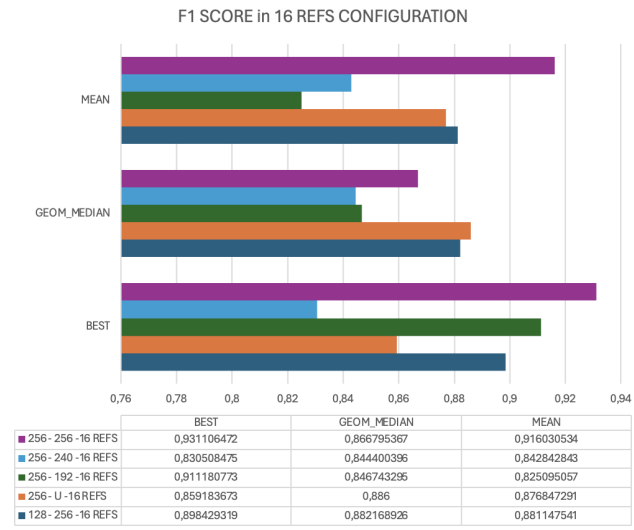


	BEST	GEOM_MEDIAN	MEAN
256 - 16 REFS	0,8708	0,6344	0,6193
128 - 16 REFS	0,2624	0,5903	0,5926

Figure 5.4: F1 Score with 16 References (PREC=1)

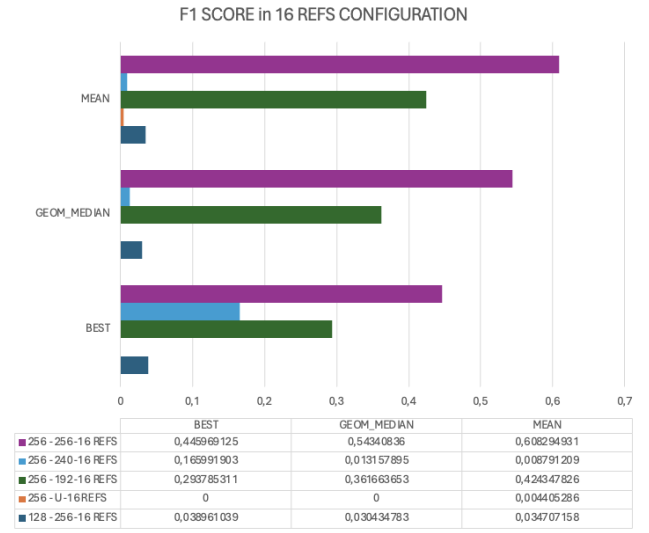
5.3.4 Distillation Knowledge and SV Dense Neural Networks

The models seen in the previous chapter were Convolutional; however, as introduced before, Syntiant NDP101 does not support this type of models, but only dense neural networks. Because of this reason, the distillation knowledge introduced in **Subsection 3.2.2** should be applied to these two convolutional models. The test models with the new create dense configurations consist in the ones introduced in **Section 5.1**. It is important to remark that the model 256-256 does not fit inside Syntiant NDP101 and it is only taken as reference, because it should be an optimal case that should outperform the others proposed. This should happen because of no neuron's reduction and having the most trainable parameters that should mimic better than the others the behavior of the Convolutional Neural Network. The training of this Dense Model is performed by parsing training input samples to the original model that will no modify its training parameters, then the student model tries to mimic the output of the master and the training parameters will modify according to the accuracy which will be cosine similarity and the loss 1-cosine similarity. The choice of this is due to achieve the objective to have the two models' output as close as possible. For possible future optimization of the models may be performed a layer per layer aware training, but to verify if the model can work with a standard distillation knowledge it was decided to focus on verify the possible versions deployable instead of optimizing them with more consistent solutions. In average, the cosine similarity of the output vectors is around 87,5%. During the distillation of every models, the dataset of Speaker Verification was given in input to them. It contains only the samples that passed KWS, Sheila words above 80% probability, so 2014 samples (453 positives and 1761 negatives).



	BEST	GEOM_MEDIAN	MEAN
256 - 256 -16 REFS	0,931106472	0,866795367	0,916030534
256 - 240 -16 REFS	0,830508475	0,844400396	0,842842843
256 - 192 -16 REFS	0,911180773	0,846743295	0,825095057
256 - U -16 REFS	0,859183673	0,886	0,876847291
128 - 256 -16 REFS	0,898429319	0,882168926	0,881147541

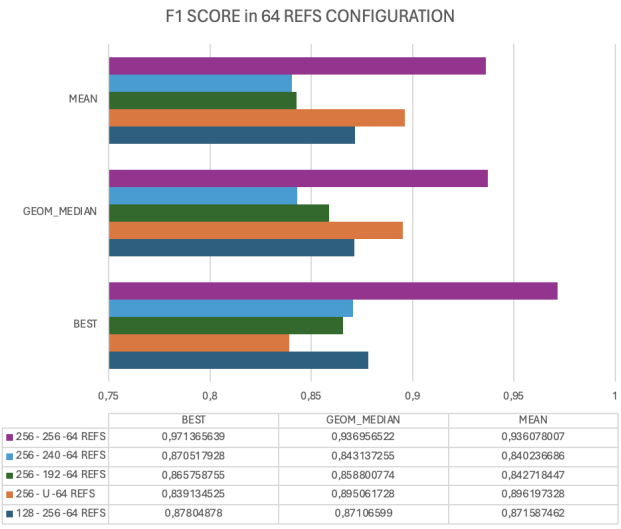
Figure 5.5: F1 Score with 16 References (<EER)



	BEST	GEOM_MEDIAN	MEAN
256 - 256 -16 REFS	0,445969125	0,54340836	0,608294931
256 - 240 -16 REFS	0,165991903	0,013157895	0,008791209
256 - 192 -16 REFS	0,293785311	0,361663653	0,424347826
256 - U -16 REFS	0	0	0,004405286
128 - 256 -16 REFS	0,038961039	0,030434783	0,034707158

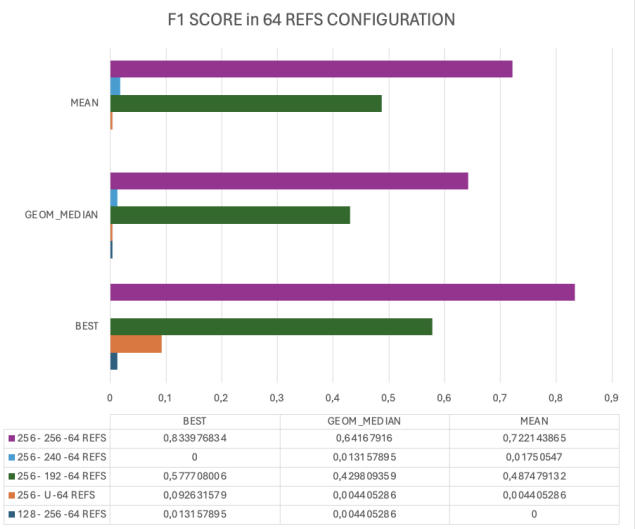
Figure 5.6: F1 Score with 16 References (PREC=1)

The results obtained by the five models in both minimum EER and precision equal to 1 case, can be seen respectively in **Table A.3** and in **Table A.4**. Starting with minimum EER case, some patterns from Convolutional models are recurring, as fact, single reference give poor performances as are still unsufficient to be a solid structure. Going up with the number the accuracy and F1 Score continue increasing, arriving at 64 references that starts manifesting in some models so inefficiency in recall like saw in Convolutional results. As it is logical the EER performance decreases significantly with more references. According, to the reference perspective the best case should be the 16 references one, which results are displayed in **Table 5.5**. From that it is seeable that BEST method achieves the best results as expected, GEOM_MEDIAN gives more stable ones, minimizing the impact of some outliers and generally performs slightly better than MEAN method except in 256-256 case, which is not applicable in this case for Memory Flash space reasons, so overall to save space, even though it is a more energy consumption algorithm is recommended more GEOM_MEDIAN. However, as said in **Subsection 5.3.2**, using EER it is the threshold in which it is minimized maximizing TP and minimizing FP, but it does not guarantee security, so it is required a study on no FP tolerance version, which results are in **Figure 5.6**. In this case, happened that there where cases in which the TP were 0, so to avoid not useful data, the threshold was decreased, leading in same cases in having a precision not equal to 1. These cases can be recognized having a missing column in **Figure 5.6** and having 0 as value in the detailed table under the figure. In this case, in many configurations can be seen an high false negative rate, but there are 2 critical performance degradation. The first one is an overfitting case in BEST Method, as fact GEOM_MEDIAN outperforms it, underlying that at his threshold these distilled models recognize more outliers. It may be good in sort of a way, but at the same time the false negative rates are pretty high and this leads in making a choose between model optimization or choosing a higher number of reference d-vectors. Another necessity to make this choose are the models with good results, because the best performing one consists in 256-256 configuration achieving with BEST 45%, with GEOM_MEDIAN 54% and with MEAN 61%, which are the best ones, but it means the recall is even less (around 50% in MEAN case) and the model does not fit on Syntiant. Increasing the references number to 64 references, are obtained the results in **Table 5.7** and **Table 5.8**. In EER minimization case, the results for not optimized or unbalanced versions have slightly worse results, instead in 256-256 which is the most consist one it should perform better, but the other are still good results which can be used in a general purpose use scenario. Instead, in no FP tolerance case, the problems of 16 references configurations are resolved, having as expected 256-256 that performs as the best one, having in BEST case over 80% of F1 Score, indicating a Recall around 70%, which can be acceptable, however the model does not fit on Syntiant, so if a user have to use a security approach on Syntiant should use 256-192 model, which in BEST case has a F1 score around 58%, but a recall around 41% which is not optimal in size terms and in performance, but using these unoptimized models these are the best results obtainable to obtain a precision of 1.



	BEST	GEOM_MEDIAN	MEAN
256 - 256 -64 REFS	0,971365639	0,936956522	0,936078007
256 - 240 -64 REFS	0,870517928	0,843137255	0,840236686
256 - 192 -64 REFS	0,865758755	0,858800774	0,842718447
256 - U -64 REFS	0,839134525	0,895061728	0,896197328
128 - 256 -64 REFS	0,87804878	0,87106599	0,871587462

Figure 5.7: F1 Score with 64 References (<EER)



	BEST	GEOM_MEDIAN	MEAN
256 - 256 -64 REFS	0,833976834	0,64167916	0,722143865
256 - 240 -64 REFS	0	0,013157895	0,01750547
256 - 192 -64 REFS	0,577708006	0,429809359	0,487479132
256 - U -64 REFS	0,092631579	0,004405286	0,004405286
128 - 256 -64 REFS	0,013157895	0,004405286	0

Figure 5.8: F1 Score with 64 References (PREC=1)

6 Conclusion & Future Work

6.1 Technical Limitations and Trade-offs

The limited information regarding Syntiant's documentation limited the effective model testing because of the inability to effectively quantize the model to 4-int weights. The device would be optimal for such usages, but because of legal limitations and hardware limitations, such as the support of only DNN architectures, the SV capabilities are limited. However, the models obtained both convolution and distilled may be still used on other TinyML microcontrollers, in case they meet the size requirements and have an MFE block generator integrated in it, because the library and spectrogram computation occupies almost 20KB, which may not be optimal and with TinyML can exhaust memory, if an external memory is not used for dataset saving. So, introducing an SD card, even though it requires a longer elaboration time for the output, may be the best solution.
During the discussion of the results, various trade-offs emerged in the choice of a model configuration with respect to the other:
• Security vs User Experience: the choosing of the threshold is crucial in determining how a model satisfies the user objective. Choosing a too low threshold leads to having a high rate of both TP and FP, accepting all samples, but, on the other hand, choosing a too low one leads to having a high rate of TN and FN, accepting no samples.
• Model Size vs Complexity: Convolutional and dense layers, which are pretty different in the operations they do, were discussed in this thesis. The CNN performs more articulated and power requirement operations, but, on the other hand, they are small sized in training parameters terms, instead DNN performs easier operations, but they require more parameters to describe harder patterns, occupying more space.
• Cosine Similarity vs Dataset Space: 3 methods were presented to handle reference d-vectors. Bestmatching is the technique that requires N references that occupy xN times the space of geometrical median and mean techniques. However, the cosine similarity has higher values with the bestmatching, instead of the other two, because they are an average sample and not exactly a recorded sample.

6.2 Key Contributions and Future Directions

This thesis provides a complete software pipeline of the system that is easily modifiable to integrate other audio neural networks. The models are optimized only in terms of performance; meanwhile, code quantization in GitHub repository[9], is proposed. It gives different results by Tensorflow version, so this feature was not dissected during the analysis. Compared with the solution TinySV proposed[22][21], it was proposed a solution with 128-sized output was proposed to reduce the dimension of the model inspired by that solution. It turned out not being as good as the original, but still usable in the general-purpose case, but it is to avoid in the perfect precision case. From that a distillation method of the models was proposed to simply the operations computation proposing various alternatives compatible and deployable (if access to Syntiant SDK is granted) with Syntiant NDP101. The results of this work may still be used on other TinyML devices, and if some company can access the SDK, the work that was done may help in generating a compatible SV model directly on the device. In addition to this, it can be integrated in a bigger system that may unlock a door or give a precise response depending on the case and the threshold used. The code is ready and on paper according to the results obtained using this One-Time Training Approach with Speaker Verification on Syntiant NDP101 is possible, but as of now, because of means limitations cannot be tested. In the end, many focus in the future on this project should be exploring hybrid architectures that combine the efficiency of TinySV with larger models accuracy and precision and build a fully compatible Syntiant NDP101 integration.

Bibliography

[1] Syntiant edge impulse. https://www.eetree.cn/wiki/_media/syntiant_tinyml_tutorial_mac_.pdf, 2021. Last Access 19/06/2025.

[2] Neural network definition and components. https://s.mriquestions.com/what-is-a-neural-network.html, 2024. Last Access 19/06/2025.

[3] Will McDonald Atul Gupta Mallik Moturi Alireza Yousefi, Luiz Franca-Neto and David Garrett. The intelligence of things enabled by syntiant's tinyml board. https://cms.tinyml.org/wp-content/uploads/talks2022/Yousefi-Alireza-hardware-FINAL.pdf. Last Access 19/06/2025.

[4] Kenneth Chan, Paul Matthews, and Kamran Munir. A framework for the estimation of air quality by applying meteorological images: Colours-of-the-wind (cold). *Environments*, 10(12), 2023. Last Access 19/06/2025.

[5] Neuhaus Christian. Code experimental implementation on ndp101. https://github.com/happychriss/Goodwatch_NDP101_SpeeechRecognition, 2021. Last Access 19/06/2025.

[6] Neuhaus Christian. Ndp101 experimental implementation on ndp101. https://44-2.de/syntiant-ndp-101-always-on-low-power-speech-recognition/, 2022. No other resources that corroborates data listed, Last Access 19/06/2025.

[7] PortAudio Community. Portaudio cross-platform audio i/o library. https://www.portaudio.com. Last Access 19/06/2025.

[8] Matteo Frigo and Steven G. Johnson. The design and implementation of FFTW3. *Proceedings of the IEEE*, 93(2):216–231, 2005. Special issue on "Program Generation, Optimization, and Platform Adaptation".

[9] Matteo Gottardelli. Syntiant ndp101 speaker verification solution proposal. https://github.com/Gotta003/Syntiant-NDP101-Speaker-Verification-Thesis. Last Access 19/06/2025.

[10] Jianping Gou, Baosheng Yu, Stephen Maybank, and Dacheng Tao. Knowledge distillation: A survey. 06 2020. Last Access 19/06/2025.

[11] Kyuyeon Hwang, Minjae Lee, and Wonyong Sung. Online keyword spotting with a character-level recurrent neural network. https://arxiv.org/abs/1512.08903, 2015. Last Access 19/06/2025.

[12] Edge Impulse. Audio syntiant. https://docs.edgeimpulse.com/docs/edge-impulse-studio/processing-blocks/audio-syntiant. Last Access 19/06/2025.

[13] Edge Impulse. Edge impulse syntiant tinyml deploying. https://docs.edgeimpulse.com/docs/edge-ai-hardware/mcu-+-ai-accelerators/syntiant-tinyml-board. Last Access 19/06/2025.

[14] Edge Impulse. Keyword spotting on syntiant stop/go example by edge impulse. https://docs.edgeimpulse.com/docs/run-inference/hardware-specific-tutorials/responding-to-your-voice-syntiant-rc-commands-go-stop. Last Access 19/06/2025.

[15] Edge Impulse. Audio classification - keyword spotting. https://studio.edgeimpulse.com/public/499022/latest, 2024. Apache 2.0, Last Access 19/06/2025.

[16] Edge Impulse. Edge impulse block processing repository. https://github.com/edgeimpulse/processing-blocks, 2025. Last Access 19/06/2025.

[17] Edge Impulse. Firmware syntiant tinyml github repository. https://github.com/edgeimpulse/firmware-syntiant-tinyml/tree/master, 2025. Last Access 19/06/2025.

[18] Ying Liu, Yan Song, Ian McLoughlin, Lin Liu, and Li-rong Dai. *An Effective Deep Embedding Learning Method Based on Dense-Residual Networks for Speaker Verification*. 2021. Last Access 19/06/2025.

[19] Sebastian Neumayer, Max Nimmer, Simon Setzer, and Gabriele Steidl. On the robust pca and weiszfeld's algorithm. https://arxiv.org/abs/1902.04292, 2019. Last Access 19/06/2025.

[20] Keiron O'shea and Ryan Nash. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015. Last Access 19/06/2025.

[21] Massimo Pavan, Gioele Mombelli, Francesco Sinacori, and Manuel Roveri. Tinysv d-vector extractor github reference. https://github.com/AI-Tech-Research-Lab/TinySV, 2023. Last Access 19/06/2025.

[22] Massimo Pavan, Gioele Mombelli, Francesco Sinacori, and Manuel Roveri. Tinysv: Speaker verification in tinyml with on-device learning. 2025. Last Access 19/06/2025.

[23] Rukshan Pramoditha. Two or more layers deep model. https://medium.com/data-science-365/two-or-more-hidden-layers-deep-neural-network-architecture-9824523ab903, 2022. Last Access 19/06/2025.

[24] Syntiant. General description of ndp101 device. https://www.syntiant.com/ndp101#data_sheet. Last Access 19/06/2025.

[25] Syntiant. hardware_ndp101. https://www.syntiant.com/hardware. Last Access 19/06/2025.

[26] Rishabh Tak, Dharmesh Agrawal, and Hemant Patil. *Novel Phase Encoded Mel Filterbank Energies for Environmental Sound Classification*. 11 2017. Last Access 19/06/2025.

[27] Tensorflow. Post training quantization tensorflow lite. https://www.tensorflow.org. Last Access 19/06/2025.

[28] Tensorflow. Quantization aware training tensorflow lite. https://www.tensorflow.org. Last Access 19/06/2025.

[29] Kite Thomas. Understanding pdm digital audio. https://users.ece.utexas.edu/ bevans/courses/rtdsp/lectures/10_Data_Conversion/AP_Understanding_PDM_Digital_Audio.pdf, 2012. Last Access 19/06/2025.

[30] TinyML. Tinyml 2021 summit presentation syntiant ndp120. https://cms.tinyml.org/wp-content/uploads/summit2021/tinyMLSummit2021d1_Awards_Syntiant.pdf, 2021. Last Access 19/06/2025.

[31] Hakeem Ullah, Aisha M. Alqahtani, Mehreen Fiza, Kashif Ullah, Muhmmad Shoaib, Ilyas Khan, Aasim Ullah Jan, and Abdoalrahman S.A. Omer. Stability analysis of mhd jeffery–hamel flow using artificial neural network. *International Journal of Thermofluids*, 24:100834, 2024. Last Access 19/06/2025.

[32] Daniel Povey Vassil Panayotov, Guoguo Chen and Sanjeev Khudanpur. Open dataset speech with various speakers and many hours speaking. https://www.openslr.org/12, 2015. Last Access 19/06/2025.

[33] P. Warden. Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition. *ArXiv e-prints*, April 2018. Last Access 19/06/2025.

[34] Xiaoxia Wu, Cheng Li, Reza Yazdani Aminabadi, Zhewei Yao, and Yuxiong He. Understanding int4 quantization for transformer models: Latency speedup, composability, and failure cases. https://arxiv.org/abs/2301.12017, 2023. Last Access 19/06/2025.

Acknowledgements

I've heard it said that people come into our lives for a reason, bringing something we must learn. Whether or not I fully believe that, I know I am who I am today thanks to those who stood by me and I hope I gave something back in return.

First, I want to thank Sinan, who guided me throughout this thesis with calm clarity, encouragement, and humanity. His support helped me stay grounded when things got foggy, and I'll carry his lessons forward.

To my parents, Elisa and Gianni, your love, patience, and endless support made this possible. You taught me kindness, perseverance, and how to keep going even in uncertain times. This milestone is as much yours as it is mine. To my sisters, Giulia and Daisy, one human and one with four paws: thank you for your light, strength, and for keeping our family joyful. Giulia, keep running your race through the fog—you'll find your way.

To my grandparents, both those here and those who live on in memory: thank you for your stories, strength, and presence in shaping my life.

To my colleagues — Alessandro, Daniele, Luis, Niccolò, and others — thank you for making this academic journey full of laughs, lessons, and chaos. We made it through together.

To Matteo, Cav, Fart, you changed my perspective, challenged me, inspired me, and reminded me how powerful real connection can be. Each of you brought something unforgettable into my life.

Thanks to everyone who got into the car of my existence and supported me in this apotheosis of path —even if only for a stretch of the ride. Some stayed, some left, but all made it memorable.

Finally, thank to every person and moment that shaped me: from childhood friends , teachers, extended family, and unexpected allies. You helped build the person I am today, and I wouldn't change a thing. To those who still walk with me: may we keep flying, even if solo, because in the end, "everyone somehow has to fly and defying the gravity".

Appendix A Complete Test Results

A.1 Convolutional Models (EER Threshold)

Type	Method	N° Refs	Acc	Prec	Recall	F1 Score	EER	AUC	Thres
128(MY)	BEST	1	0.7989	0.5057	0.6858	0.5822	0.3142	0.5005	0.5
		8	0.9007	0.6864	0.9348	0.7916	0.1079	0.9241	0.6
		16	0.9145	0.7114	0.9588	0.8168	0.0965	0.9650	0.65
		64	0.9526	0.8121	0.9616	0.8806	0.0494	0.9842	0.7
	GEOM	1	0.7989	0.5057	0.6858	0.5822	0.3142	0.5005	0.5
		8	0.9238	0.7551	0.9213	0.8300	0.0786	0.9340	0.6
		16	0.9200	0.7326	0.9405	0.8236	0.0852	0.9505	0.6
		64	0.9535	0.8407	0.9182	0.8777	0.0818	0.9633	0.65
	MEAN	1	0.7989	0.5057	0.6858	0.5822	0.3142	0.5005	0.5
		8	0.9252	0.7583	0.9236	0.8328	0.0764	0.9359	0.6
		16	0.9204	0.7331	0.9428	0.8248	0.0852	0.9525	0.65
		64	0.9526	0.8353	0.9207	0.8759	0.0793	0.9639	0.65
256[22]	BEST	1	0.6751	0.3321	0.5847	0.4234	0.4159	0.4787	0.5
		8	0.9429	0.7874	0.9820	0.8740	0.0670	0.9878	0.65
		16	0.9745	0.8944	0.9886	0.9391	0.0290	0.9968	0.7
		64	0.9805	0.9125	0.9872	0.9484	0.0210	0.9974	0.75
	GEOM	1	0.6751	0.3321	0.5847	0.4234	0.4159	0.4787	0.6
		8	0.9198	0.7454	0.9146	0.8214	0.0854	0.9366	0.6
		16	0.8917	0.6628	0.9268	0.7729	0.1170	0.9408	0.6
		64	0.9610	0.8434	0.9642	0.9000	0.0398	0.9800	0.65
	MEAN	1	0.6751	0.3321	0.5847	0.4234	0.4159	0.4787	0.5
		8	0.8395	0.5658	0.8786	0.6884	0.1704	0.9511	0.65
		16	0.9290	0.7513	0.9611	0.8438	0.0789	0.9559	0.6
		64	0.9577	0.8289	0.9667	0.8926	0.044	0.9824	0.65
COMP	BEST	1	12.38	17.36	10.18	15.87	10.18	2.36	
		8	-4.22	-10.09	-4.72	-8.24	-4.09	-6.37	
		16	-6.01	-18.30	-2.97	-12.24	-6.76	-3.18	
		64	-2.79	-10.04	-2.56	-6.78	-2.84	-1.32	
	GEOM	1	12.38	17.36	10.18	15.87	10.18	2.36	
		8	3.45	9.11	0.67	6.06	3.83	-0.05	
		16	2.59	6.43	1.37	4.70	2.90	0.80	
		64	0.65	5.04	-4.60	0.91	-2.51	-1.46	
	MEAN	1	12.38	17.36	10.18	15.87	10.18	2.36	
		8	2.72	8.31	-2.92	4.25	3.94	-1.62	
		16	-2.27	-10.90	6.41	-3.52	3.61	-0.26	
		64	0.46	4.28	-4.60	0.49	-2.31	-1.69	

Table A.1: SV Convolutional Models (EER Threshold)

A.2 Convolutional Models (Minimizing Only False Positive)

Type	Method	N° Refs	Acc	Prec	Recall	F1 Score	EER	AUC	Thres
128(MY)	BEST	1	0.8156	1	0.0973	0.1774	0.9026	0.5023	0.7
		8	0.8173	1	0.0944	0.1725	0.9056	0.9241	0.85
		16	0.8312	1	0.1510	0.2624	0.8490	0.9650	0.85
		64	0.8815	1	0.3478	0.5161	0.6522	0.9843	0.85
	GEOM	1	0.8156	1	0.0973	0.1774	0.9026	0.5023	0.7
		8	0.8676	1	0.3438	0.5117	0.6562	0.9340	0.8
		16	0.8844	1	0.4188	0.5903	0.5812	0.9505	0.8
		64	0.8587	1	0.2225	0.3640	0.7775	0.9633	0.85
	MEAN	1	0.8156	1	0.0973	0.1774	0.9026	0.5023	0.7
		8	0.8694	1	0.3528	0.5216	0.6472	0.9340	0.8
		16	0.8849	1	0.4211	0.5926	0.5789	0.9505	0.8
		64	0.8583	1	0.2200	0.3606	0.7801	0.9633	0.85
256[22]	BEST	1	0.8269	1	0.1526	0.2649	0.8473	0.5023	0.8
		8	0.9057	1	0.5326	0.6950	0.4674	0.9359	0.8
		16	0.9545	1	0.7712	0.8708	0.2288	0.9525	0.8
		64	0.9466	1	0.7059	0.8276	0.2941	0.9639	0.85
	GEOM	1	0.8269	1	0.1526	0.2649	0.8473	0.4787	0.8
		8	0.8840	1	0.4247	0.5962	0.5753	0.9878	0.8
		16	0.8935	1	0.4645	0.6344	0.5355	0.9969	0.8
		64	0.8973	1	0.4348	0.6061	0.5652	0.9974	0.8
	MEAN	1	0.8269	1	0.1527	0.2649	0.8473	0.4787	0.8
		8	0.8785	1	0.3978	0.5691	0.6022	0.9345	0.8
		16	0.8903	1	0.4485	0.6193	0.5515	0.9425	0.8
		64	0.8922	1	0.4066	0.5782	0.5934	0.9780	0.8
COMP	BEST	1	-1.13	0.00	-5.53	-8.75	-5.53	2.36	
		8	-8.84	0.00	-43.82	-52.25	-43.82	-6.37	
		16	-12.33	0.00	-62.01	-60.84	-62.01	-3.18	
		64	-6.51	0.00	-35.81	-31.15	-35.81	-1.32	
	GEOM	1	-1.13	0.00	-5.53	-8.75	-5.53	2.36	
		8	-1.63	0.00	-8.09	-8.45	-8.09	-0.05	
		16	-0.91	0.00	-4.58	-4.41	-4.58	0.80	
		64	-3.86	0.00	-21.23	-24.20	-21.23	-1.46	
	MEAN	1	-1.13	0.00	-5.53	-8.75	-5.53	2.36	
		8	-0.91	0.00	-4.49	-4.75	-4.49	-1.62	
		16	-0.55	0.00	-2.75	-2.67	-2.75	-0.26	
		64	-3.99	0.00	-18.67	-21.76	-18.67	-1.69	

Table A.2: SV Convolutional Models (Minimizing Only False Positive)

A.3 Dense Models (EER Thresholds)

Table A.3: SV Dense Models (EER Threshold)

Model	Method	N° Refs	Acc	Prec	Recall	F1	EER	AUC	Thres
128-256	BEST	1	0.9322	0.7770	0.9382	0.8500	0.0693	0.8083	0.55
		8	0.9033	0.7268	0.8455	0.7816	0.1545	0.9256	0.7
		16	0.9562	0.8546	0.9470	0.8984	0.0530	0.9695	0.75
		64	0.9481	0.8449	0.9139	0.8780	0.0861	0.9684	0.8
	GEOM	1	0.9322	0.7770	0.9382	0.8500	0.0693	0.8083	0.55
		8	0.9372	0.7985	0.9271	0.8580	0.0728	0.9164	0.65
		16	0.9490	0.8360	0.9338	0.8822	0.0662	0.9221	0.65
		64	0.9426	0.8064	0.9470	0.8711	0.0585	0.9285	0.65
	MEAN	1	0.9322	0.7770	0.9382	0.8500	0.0693	0.8083	0.55
		8	0.9354	0.7913	0.9293	0.8548	0.0706	0.9201	0.65
		16	0.9476	0.8222	0.9492	0.8814	0.0528	0.9318	0.65
		64	0.9426	0.8041	0.9514	0.8716	0.0596	0.9299	0.65
256U	BEST	1	0.8347	0.5598	0.8985	0.6898	0.1817	0.8813	0.75
		8	0.9435	0.7971	0.9713	0.8756	0.0636	0.9806	0.8
		16	0.9377	0.7989	0.9294	0.8592	0.0706	0.9743	0.8
		64	0.9228	0.7311	0.9845	0.8391	0.0931	0.9776	0.8
	GEOM	1	0.8347	0.5598	0.8985	0.6898	0.1817	0.8813	0.75
		8	0.9553	0.8430	0.9603	0.8978	0.0460	0.9827	0.8
		16	0.9485	0.8098	0.9779	0.8860	0.0591	0.9837	0.8
		64	0.9539	0.8382	0.9603	0.8951	0.0477	0.9820	0.75
	MEAN	1	0.8347	0.5598	0.8985	0.6898	0.1817	0.8813	0.75
		8	0.9593	0.8682	0.9448	0.9049	0.0552	0.9818	0.8
		16	0.9435	0.7918	0.9823	0.8768	0.0664	0.9809	0.8
		64	0.9544	0.8385	0.9625	0.8962	0.0477	0.9817	0.8
256-192	BEST	1	0.8098	0.5220	0.8389	0.6435	0.1976	0.7227	0.5
		8	0.9517	0.8366	0.9492	0.8893	0.0508	0.9767	0.8
		16	0.9616	0.8658	0.9625	0.9112	0.0386	0.9884	0.8
		64	0.9377	0.7739	0.9823	0.8658	0.0738	0.9885	0.8
	GEOM	1	0.8098	0.5220	0.8389	0.6435	0.1976	0.7227	0.5
		8	0.8022	0.5110	0.7704	0.6144	0.2296	0.7819	0.65
		16	0.9277	0.7479	0.9757	0.8467	0.0846	0.9797	0.7
		64	0.9341	0.7642	0.9801	0.8588	0.0778	0.9830	0.7
	MEAN	1	0.8098	0.5220	0.8389	0.6435	0.1976	0.7227	0.5
		8	0.8035	0.5120	0.8477	0.6384	0.2078	0.8422	0.65
		16	0.9169	0.7245	0.9581	0.8251	0.0937	0.9727	0.7
		64	0.9268	0.7522	0.9581	0.8427	0.0812	0.9778	0.7
256-240	BEST	1	0.8180	0.5392	0.7594	0.6306	0.2406	0.8094	0.65
		8	0.9359	0.8008	0.9139	0.8536	0.0861	0.9548	0.75
		16	0.9187	0.7241	0.9735	0.8305	0.0954	0.9763	0.75
		64	0.9413	0.7931	0.9646	0.8705	0.0647	0.9800	0.8
	GEOM	1	0.8180	0.5392	0.7594	0.6306	0.2406	0.8094	0.65
		8	0.7778	0.4744	0.7969	0.5947	0.2271	0.7585	0.6

Continued on next page

Table A.3 – continued from previous page

Model	Method	N° Refs	Acc	Prec	Recall	F1	EER	AUC	Thres
256-240	GEOM	16	0.9291	0.7662	0.9404	0.8444	0.0738	0.9579	0.7
		64	0.9277	0.7584	0.9492	0.8431	0.0778	0.9587	0.7
	MEAN	1	0.8180	0.5392	0.7594	0.6306	0.2406	0.8094	0.65
		8	0.8722	0.6221	0.9558	0.7537	0.1493	0.9224	0.65
		16	0.9291	0.7711	0.9294	0.8428	0.0710	0.9590	0.7
		64	0.9268	0.7594	0.9404	0.8402	0.0767	0.9582	0.7
256-256	BEST	1	0.6576	0.3470	0.7638	0.4772	0.3697	0.6119	0.5
		8	0.9589	0.8935	0.9073	0.9003	0.0927	0.9858	0.8
		16	0.9702	0.8832	0.9845	0.9311	0.0335	0.9919	0.8
		64	0.9883	0.9692	0.9735	0.9714	0.0265	0.9957	0.85
	GEOM	1	0.6576	0.3470	0.7638	0.4772	0.3697	0.6119	0.5
		8	0.8410	0.5716	0.8896	0.6960	0.1715	0.9111	0.65
		16	0.9377	0.7702	0.9912	0.8668	0.0761	0.9829	0.7
		64	0.9738	0.9229	0.9514	0.9370	0.0486	0.9868	0.75
	MEAN	1	0.6576	0.3470	0.7638	0.4772	0.3697	0.6119	0.5
		8	0.9029	0.7179	0.8653	0.7848	0.1347	0.9382	0.7
		16	0.9652	0.9052	0.9272	0.9160	0.0728	0.9852	0.75
		64	0.9733	0.9191	0.9536	0.9361	0.0464	0.9887	0.75

A.4 Dense Models (Minimizing Only False Positive)

Table A.4: SV Dense Models (Minimizing Only False Positive)

Model	Method	N° Refs	Acc	Prec	Recall	F1	EER	AUC	Thres
128-256	BEST	1	0.7967	0.8000	0.0088	0.0175	0.9912	0.8083	0.85
		8	0.8026	0.9444	0.0375	0.0722	0.9625	0.9256	0.9
		16	0.7994	1	0.0199	0.0390	0.9801	0.9695	0.95
		64	0.7967	1	0.0066	0.0131	0.9934	0.9684	0.95
	GEOM	1	0.7967	0.8	0.0088	0.0175	0.9912	0.8083	0.85
		8	0.7967	0.8	0.0088	0.0175	0.9912	0.9164	0.9
		16	0.7985	1	0.0154	0.0304	0.9845	0.9221	0.9
		64	0.7958	1	0.0022	0.0044	0.9978	0.9285	0.95
	MEAN	1	0.7967	0.8	0.0088	0.0175	0.9912	0.8083	0.85
		8	0.7967	0.8	0.0088	0.0175	0.9912	0.9201	0.9
		16	0.7990	1	0.0177	0.0347	0.9823	0.9318	0.9
		64	0.7995	0.9091	0.0221	0.0347	0.9779	0.9299	0.9
256U	BEST	1	0.7967	1	0.0066	0.0132	0.9934	0.8813	0.85
		8	0.7972	1	0.0088	0.0175	0.9912	0.9806	0.9
		16	0.8907	0.9607	0.4856	0.6452	0.5143	0.9743	0.85
		64	0.8053	1	0.0486	0.0926	0.9514	0.9776	0.9
	GEOM	1	0.7967	1	0.0066	0.0132	0.9934	0.8813	0.85
		8	0.9205	0.9759	0.6269	0.7634	0.3731	0.9827	0.85
		16	0.9336	0.9722	0.6954	0.8108	0.3046	0.9837	0.85
		64	0.7958	1	0.0022	0.0044	0.9978	0.9820	0.9

Continued on next page

Table A.4 – continued from previous page

Model	Method	N° Refs	Acc	Prec	Recall	F1	EER	AUC	Thres
256U	MEAN	1	0.7967	1	0.0066	0.0132	0.9934	0.8813	0.85
		8	0.8925	0.9909	0.4790	0.6458	0.5210	0.9818	0.85
		16	0.7958	1	0.0022	0.0044	0.9978	0.9809	0.9
		64	0.7958	1	0.0022	0.0044	0.9978	0.9817	0.9
256-192	BEST	1	0.8415	1	0.2252	0.3676	0.7748	0.7227	0.75
		8	0.8270	1	0.1545	0.2677	0.8455	0.9767	0.9
		16	0.8306	1	0.1722	0.2938	0.8278	0.9884	0.9
		64	0.8785	1	0.4062	0.5777	0.5938	0.9885	0.9
	GEOM	1	0.8415	1	0.2252	0.3676	0.7748	0.7227	0.75
		8	0.8103	1	0.0728	0.1358	0.9272	0.7819	0.9
		16	0.8406	1	0.2208	0.3617	0.7792	0.9797	0.85
		64	0.8514	1	0.2737	0.4298	0.7263	0.9830	0.85
	MEAN	1	0.8415	1	0.2252	0.3676	0.7748	0.7227	0.75
		8	0.8098	1	0.0706	0.1320	0.9293	0.8422	0.9
		16	0.8505	1	0.2693	0.4243	0.7307	0.9727	0.85
		64	0.8613	1	0.3223	0.4875	0.6777	0.9778	0.85
256-240	BEST	1	0.7977	1	0.0110	0.0218	0.9890	0.8094	0.9
		8	0.7967	0.8000	0.0088	0.0175	0.9912	0.9549	0.9
		16	0.8139	1	0.0905	0.1660	0.9095	0.9763	0.9
		64	0.8482	0.9916	0.2605	0.4126	0.7395	0.9800	0.9
	GEOM	1	0.7977	1	0.0110	0.0218	0.9890	0.8094	0.9
		8	0.7958	1	0.0022	0.0044	0.9978	0.7585	0.9
		16	0.7967	1	0.0066	0.0132	0.9934	0.9579	0.9
		64	0.7967	1	0.0066	0.0132	0.9934	0.9587	0.9
	MEAN	1	0.7977	1	0.0011	0.0218	0.9890	0.8094	0.9
		8	0.8031	1	0.0375	0.0723	0.9625	0.9224	0.85
		16	0.7963	1	0.0044	0.0088	0.9956	0.9590	0.9
		64	0.7972	1	0.0088	0.0175	0.9912	0.9582	0.9
256-256	BEST	1	0.8342	1	0.1898	0.3191	0.8102	0.6119	0.75
		8	0.8975	1	0.4989	0.6657	0.5011	0.9858	0.9
		16	0.8541	1	0.2870	0.4460	0.7130	0.9919	0.9
		64	0.9417	1	0.7152	0.8340	0.2848	0.9957	0.9
	GEOM	1	0.8342	1	0.1898	0.3191	0.8102	0.6119	0.75
		8	0.8993	1	0.5077	0.6735	0.4922	0.9111	0.85
		16	0.8717	1	0.3731	0.5434	0.6269	0.9829	0.85
		64	0.8921	1	0.4724	0.6417	0.5276	0.9868	0.85
	MEAN	1	0.8342	1	0.1898	0.3191	0.8102	0.6120	0.75
		8	0.9101	1	0.5607	0.7185	0.4393	0.9382	0.85
		16	0.8848	1	0.4371	0.6083	0.5629	0.9852	0.85
		64	0.9110	1	0.5661	0.7221	0.4349	0.9887	0.85